Laporan Tugas Besar
Teori Bahasa Formal dan Automata
Milestone 1 - Lexical Analysis

Disusun oleh kelompok :
Kerjakan J - 5

Mahatma Brahmana	13524015
Benedict Darrel Setiawan	13524057
Raymond Jonathan Dwi Putra J	13524059
Reynard Anderson Wijaya	13524111

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Daftar Isi

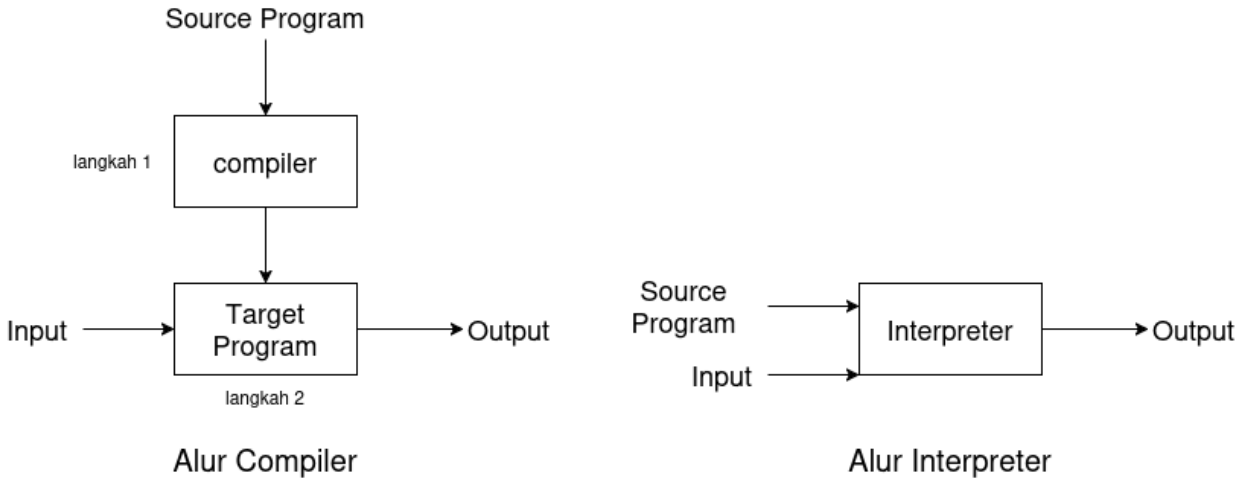
Daftar Isi **2**
BAB 1
Landasan Teori **3**
1.1 Bahasa Pemrograman dan Konversinya 3
1.2 Analisis Leksikal 4
1.3 Deterministic Finite Automata (DFA) 4
1.4 Kaitan Analisis Leksikal dengan Token dan DFA 5
BAB 2
Perancangan Program **6**
2.1 Gambaran Umum dan Pilihan Teknologi 6
2.2 Struktur Program dan Fungsi Utama 9
2.2.1 Struktur Direktori Utama 9
2.2.2 Fungsi dan Kelas Utama Program 9
2.3 Alur Kerja Program 10
BAB 3
Implementasi **14**
3.1 Implementasi Diagram Transisi DFA 14
3.1.1 Diagram DFA Start State 14
3.1.2 Diagram DFA Keyword Trie State 14
3.1.3 Diagram DFA Numerical State 18
3.1.4 Diagram DFA Symbols State 19
3.1.5 Diagram DFA String and Character State 20
3.1.6 Diagram DFA Comment State 21
3.1.7 Diagram DFA Indent State 21
3.1.8 Diagram DFA Unknown State 22
3.2 Implementasi Modul Token dan State 22
3.3 Implementasi Logika DFA (Mesin Lexer) 32
3.4 Implementasi Program Utama (Main) 35
BAB 4
Pengujian **36**
4.1 Pengujian Masukkan dan Keluaran Program 36
BAB 5
Kesimpulan dan Saran **41**
5.1 Kesimpulan 41
5.2 Saran 41
Lampiran **42**
Daftar Pustaka **43**

BAB 1
Landasan Teori

1.1 Bahasa Pemrograman dan Konversinya

Dalam pembuatan suatu program ataupun software, pembuatan tersebut dibuat dengan bahasa pemrograman tertentu untuk bisa menjalankannya. Bahasa Pemrograman adalah bahasa atau penulisan program komputer yang digunakan untuk menjalankan suatu perintah yang ingin dibuat tanpa perlu melakukan pemahaman terhadap penulisan dalam bahasa mesin. Bahasa mesin umumnya berupa bilangan biner yang berisikan 1 atau 0 yang dapat dipahami dan dijalankan oleh CPU (*Central Processing Unit*), sehingga sulit untuk dipahami oleh manusia. Bahasa pemrograman ini menjadi kemudahan dalam manusia membuat program yang ingin dan sering kali dibedakan dengan istilah *low-level* dan *high-level*. Bahasa Pemrograman *high-level* merupakan bahasa yang dirancang untuk mudah dipahami, ditulis, dan dibaca oleh manusia dalam menuliskan program yang diinginkan, sehingga banyak sekali abstraksi yang menyembunyikan detail memori dan instruksi prosesor. Sedangkan bahasa pemrograman *low-level* merupakan bahasa yang dirancang untuk dibuat dan ditulis mendekati dengan arsitektur perangkat keras komputer atau bahasa mesin, dengan sedikit atau tanpa abstraksi.

Dalam memproses atau menjalankan bahasa pemrograman itu sendiri, diperlukan pengubah untuk mengubah kode bahasa pemrograman tersebut menjadi kode bahasa mesin agar dapat dibaca dan dijalankan oleh mesin. Terdapat dua cara untuk mengubah kode bahasa pemrograman menjadi kode bahasa mesin, yaitu ***compiler*** dan ***interpreter***. *Compiler* merupakan proses dimana kode program dari bahasa pemrograman tersebut diterjemahkan semua menjadi kode mesin (dalam bentuk file e.g Object File) sebelum program dijalankan. Compiler secara umum terdiri dari 4 tahapan dalam pemrosesannya, yaitu Lexical Analysis, Syntax Analysis, Semantic Analysis dan Intermediate Code generation. *Interpreter* merupakan proses dimana kode program dieksekusi secara baris per baris secara langsung bersamaan dengan proses *input* dan *output* dari program. Berbeda dengan *compiler* yang harus membuat kode keseluruhan tanpa harus ada yang *error,* interpreter akan tetap berjalan dari baris per baris hingga terdapat baris yang *error*. Interpreter secara umum memiliki tahapan yang sama seperti pada *compiler*.



Gambar 1.1 Alur Compiler dan Interpreter

1.2 Analisis Leksikal

Analisis leksikal (*lexical analysis* atau *scanning*) adalah tahap awal dalam proses kompilasi dan interpretasi bahasa pemrograman. Pada tahap ini, *lexical analyzer*(atau *lexer*) membaca aluran karaker yang menyusun program utama dan mengelompokkannya menjadi rangkaian bermakna yang disebut *lexemes*. Setiap *lexeme* yang teridentifikasi dari aliran karakter tersebut kemudian diklasifikasikan ke dalam **token.** Token merupakan unit kecil yang memiliki arti dalam sebuah program, misalnya kata kunci (*keyword*), nama variabel (*identifier*), operator, angka, atau tanda baca dengan representasi format : `<token-name, attribute-value>`. Deretan token ini kemudian diteruskan ke fase kompilasi berikutnya, yaitu analisis sintaksis (*syntax analysis*). Di dalam struktur token tersebut, terdapat dua komponen utama, yaitu :

- `token-name` : Merupakan simbol abstrak yang merepresentasikan jenis atau kategori dari *lexeme* (seperti *keyword*, *identifier*, atau *operator*). Simbol abstrak ini adalah komponen utama yang digunakan selama proses analisis sintaksis.
- `attribute-value` : Merupakan penunjuk (*pointer*) ke sebuah entri di dalam tabel simbol (*symbol table*) untuk token tersebut. Informasi dari entri tabel simbol ini nantinya akan dibutuhkan untuk tahap selanjutnya, yaitu analisis semantik dan pembuatan kode.

Merujuk pada buku *Compilers: Principles, Techniques, and Tools*, tahapan ini melibatkan tiga konsep utama yang saling berkaitan :

- **Pola (*pattern*) :** Aturan atau deskripsi logis yang menentukan struktur deretan karakter untuk membentuk suatu token tertentu.
- **Leksem (*Lexeme*) :** Deretan karakter aktual pada *source code* yang cocok dengan suatu pola dan diidentifikasi sebagai instansi dari sebuah token.
- **Token :** Kesatuan unit bermakna `<token-name, attribute-value>` yang dihasilkan dari suatu *lexeme*.

Proses analisis leksikal juga menyederhanakan kode sumber dengan membuang bagian yang tidak memiliki makna komputasi secara langsun, seperti spasi kosong (*whitespace*), dan tab.

Agar proses pembacaan karakter dan pencocok pola tersebut dapat dijalankan secara sistematis dan otomatis oleh komputer, aturan- aturan leksikal diimplementasikan ke dalam bentuk automata terbatas. Khususnya, pola-pola pembentukan token dimodelkan menggunakan *Deterministic Finite Automata* (DFA), sehingga setiap karakter yang dibaca memiliki transisi *state* yang pasti dan tidak ambigu dalam menentukan token.

1.3 Deterministic Finite Automata (DFA)

Deterministic finite automata atau DFA adalah model komputasi matematis yang digunakan untuk mengenali pola dalam suatu string masukan. DFA didefinisikan secara formal sebagai 5-tuple $(Q, \Sigma, \delta, q_0, F)$, di mana Q adalah himpunan state yang terbatas, merepresentasikan seluruh kondisi yang mungkin dimiliki oleh mesin selama proses pembacaan input. Σ adalah himpunan simbol masukan yang disebut alfabet, bisa terdiri dari huruf, angka, maupun karakter lainnya. δ adalah fungsi transisi dengan bentuk $\delta: Q \times \Sigma \rightarrow Q$, yang menentukan ke state mana mesin akan berpindah ketika berada pada suatu state tertentu dan membaca simbol tertentu dari input. q_0 adalah state awal, yaitu kondisi di mana mesin mulai beroperasi sebelum membaca simbol pertama. Dan terakhir, F adalah himpunan state penerima (*accept* state), yaitu state-state yang menandakan bahwa string yang telah dibaca merupakan string yang valid atau diterima oleh mesin. Sifat 'deterministik' pada DFA berarti bahwa untuk setiap pasangan state dan simbol masukan, terdapat fungsi transisi δ yang menghasilkan tepat satu state tujuan, tidak ada

percabangan ganda maupun transisi kosong seperti yang dijumpai dalam NFA (Non-deterministic Finite Automaton). Hal ini menjamin bahwa jalur komputasi DFA selalu tungga, sehingga tidak ada ambiguitas dalam proses pengenalan string. DFA dikatakan menerima (*accept*) suatu string apabila setelah seluruh simbol dibaca, mesin berakhir pada salah satu state dalam himpunan F. Sebaliknya, jika mesin berakhir pada state di luar F, maka string tersebut akan ditolak (*reject*). Kumpulan seluruh string yang diterima oleh suatu DFA disebut bahasa regular (*regular language*), yang merupakan kelas bahasa paling sederhana dalam hirarki Chomsky.

1.4 Kaitan Analisis Leksikal dengan Token dan DFA

Analisis leksikal, token, dan DFA merupakan tiga konsep yang saling berkaitan erat dalam proses kerja sebuah kompilator. Seperti yang telah dibahas pada sub bab sebelumnya, analisis leksikal bertugas membaca karakter-karakter mentah dari kode sumber secara sekuensial dan mengelompokkannya menjadi satuan-satuan bermakna yang disebut token. Namun demikian, untuk dapat mengenali apakah suatu rangkaian karakter membentuk token yang valid, dibutuhkan suatu mekanisme pengenalan pola yang sistematis dan efisien. Di sinilah DFA mengambil perannya.

Setiap kategori token pada dasarnya dapat dideskripsikan menggunakan ekspresi reguler (*regular expression*). Ekspresi reguler tersebut kemudian dikonversi, baik secara langsung maupun melalui perantara DFA. Hasil dari pengkonversian ini menjadi sebuah DFA yang siap digunakan oleh lexer. Ketika lexer memproses kode sumber, ia menjalankan DFA terhadap karakter-karakter masukan satu per satu. Setiap kali DFA mencapai *accept* state, lexer mengakui bahwa rangkaian karakter yang dibaca membentuk sebuah token yang valid, sekaligus menentukan kelas atau tipe token tersebut. Proses ini kemudian diulang kembali dari karakter berikutnya hingga seluruh kode sumber habis diproses.

Dengan demikian, dapat dipahami bahwa DFA berfungsi sebagai mesin pengenal yang menjadi bagian utama proses analisis leksikal. Tanpa DFA, lexer tidak memiliki dasar formal untuk menentukan batas antar token maupun mengklasifikasikan token secara tepat dan tanpa ketidakjelasan. Hasil akhir dari proses ini adalah deretan token terstruktur yang selanjutnya diteruskan ke tahap analisis sintaks (*parsing*) untuk diproses lebih lanjut.

BAB 2
Perancangan Program

2.1 Gambaran Umum dan Pilihan Teknologi

Program Lexer dibuat menggunakan bahasa pemrograman C/C++ GNU dengan tidak menggunakan library atau tools yang membantu dalam menghasilkan lexical analyzer dan menambahkan MAKEFILE pada repository untuk mempermudah dalam melakukan mengkompilasi program yang telah dibuat. Program lexer ini menggunakan penerapan DFA yang membaca satu demi satu karakter yang terdapat pada karakter keyboard dengan memproses masukkan *source code* dalam bentuk .txt. Program kemudian memproses *source code* dan mengeluarkan *output* **daftar token** dalam bentuk format .txt.

Berikut merupakan daftar token yang akan dibuat pada bahasa pemrograman Arion :

Daftar Token			
No	**Token**	**Definisi & Keterangan**	**Contoh Pemakaian**
1	intcon	Konstanta integer	1, 3, 48
2	realcon	Konstanta bilangan riil	3.14, 26.7
3	charcon	Konstanta karakter singular, diapit dengan tanda petik tunggal (')	'j', 'k', 't'
4	string	Sekuens karakter, diapit dengan tanda petik tunggal (')	'IRK', 'TBFO'
5	notsy	Operator logika NOT (negasi)	NOT
6	plus	Operator aritmatika pertambahan	+
7	minus	Operator aritmatika pengurangan	-
8	times	Operator aritmatika perkalian	*
9	idiv	Operator aritmatika pembagian Integer	div

10	rdiv	Operator aritmatika pembagian Riil	/
11	imod	Operator aritmatika modulo	MOD
12	andsy	Operator logika AND	AND
13	orsy	Operator logika OR	OR
14	eql	equal	==
15	neq	Not equal	<>
16	gtr	Greater than	>
17	geq	Greater than or equal	>=
18	lss	Less than	<
19	leq	Less than or equal	<=
20	lparent	Left parentheses	(
21	rparent	Right parantheses	)
22	lbrack	Kurung siku kiri	[
23	rbrack	Kurung siku kanan	]
24	comma	-	,
25	semicolon	-	;
26	period	-	.
27	colon	-	:
28	becomes	Assignment operator	:=

29	constsy	Deklarasi konstanta	const
30	typesy	Deklarasi tipe data	type
31	varsy	Deklarasi variabel	var
32	functionsy	Deklarasi fungsi	function
33	proceduresy	Deklarasi prosedur	procedure
34	arraysy	Deklarasi array	array
35	recordsy	Deklarasi record	record
36	programsy	Deklarasi program	program
37	ident	Nama yang menunjukkan konstanta, tipe, variabel, prosedur, dan fungsi. Nama tersebut harus diawali dengan huruf, yang dapat diikuti oleh kombinasi huruf dan angka apa pun, dan berjumlah berapa pun. Bersifat case-insensitive	Integer, Real, Char, String h4g, identIniSangatPanjang24tetapiTetapValid
38	beginsy	-	begin
39	ifsy	-	if
40	casesy	-	case
41	repeatsy	-	repeat
42	whilesy	-	while
43	forsy	-	for
44	endsy	-	end
45	elsesy	-	else
46	untilsy	-	until

47	ofsy	-	of
48	dosy	-	do
49	tosy	-	to
50	downtosy	-	downto
51	thensy	-	then
52	comment	Dimulai dengan { atau (* (bukan di dalam tanda kutip) dan diakhiri dengan } atau *).	{ Ini komentar }, (* In komentar *)

Struktur folder yang dibuat pada pembuatan program ini dibentuk dengan membuat masing-masing folder lexical analysis, syntax analysis, semantic analysis dan intermediate code generation. Hal ini mempermudah dalam pembuatan program untuk kedepannya agar dapat dilakukan secara modular supaya komponen dapat dimodifikasi sesuai keperluan tanpa mempengaruhi komponen lainnya, serta dapat dengan mudah dijalankan dalam satu pipeline.

2.2 Struktur Program dan Fungsi Utama

Untuk memastikan kemudahan pengembangan, pembacaan kode, serta kelancaran integrasi pada tahap kompilasi selanjutnya (seperti *Syntax* dan *Semantic Analysis*), program *Lexical Analyzer* ini dirancang dengan arsitektur yang modular. Logika program dipecah dan dipisahkan ke dalam beberapa berkas berdasarkan tanggung jawab utamanya masing-masing (*Single Responsibility Principle*).

2.2.1 Struktur Direktori Utama

- **`src/`**: Merupakan direktori utama yang menampung seluruh kode sumber (*source code*) C++ pembentuk program *lexer*.
- **`doc/`** dan **`test/`**: Direktori tambahan yang direncanakan untuk menyimpan dokumen laporan serta berkas uji (*test cases*) berupa masukan dan keluaran program.
- **`Makefile`**: Skrip otomasi yang dikonfigurasi untuk mengompilasi seluruh modul C++ (seperti `main.cpp`, `lexer.cpp`, dan `token.cpp`) menjadi sebuah berkas *executable* tunggal secara efisien.

2.2.2 Fungsi dan Kelas Utama Program

Dalam direktori `src/`, kode sumber dibagi menjadi berkas eksekusi utama dan sub-direktori `lexer/` yang berisi definisi mesin komputasinya.

1. **Modul Token (`token.hpp` dan `token.cpp`)** Modul ini bertanggung jawab penuh atas definisi dan representasi struktur data token.

    - **`enum ListToken`**: Digunakan untuk mendaftarkan seluruh (52 jenis) token yang valid di dalam bahasa pemrograman Arion, yang dikelompokkan mulai dari literal, operator aritmatika, operator logika, hingga *keyword* kontrol alur program.
    - **Kelas `Token`**: Berfungsi sebagai wadah untuk menyimpan entitas token yang berhasil diidentifikasi. Kelas ini menyimpan tipe token (`ListToken`) dan nilai leksikal aktualnya dalam bentuk `std::string`. Terdapat pula metode `tokenTypeToString()` yang mereturn representasi teks dari suatu token untuk mempermudah pencetakan keluaran.
2. **Modul Lexer (`lexer.hpp` dan `lexer.cpp`)** Modul ini merupakan mesin inti yang menjalankan logika *Deterministic Finite Automata* (DFA).
    - **`enum State`**: Mendefinisikan secara komprehensif seluruh transisi *state* yang mungkin terjadi selama pembacaan aliran karakter. Ini mencakup *state* dasar seperti pembacaan angka (`INT_STATE`), string (`STRING_ONGOING_STATE`), hingga rincian transisi karakter demi karakter pembentuk *keyword* (misalnya transisi menuju `PROGRAM_STATE`).
    - **Fungsi `tokenize()`**: Merupakan fungsi utama pada modul ini yang menerima parameter berupa nama berkas (*file name*). Fungsi ini bertugas mengelola siklus DFA dengan membaca karakter satu per satu, menggeser transisi *state* sesuai karakter masukan, lalu mengembalikan hasilnya dalam bentuk senarai (`vector<Token>`).
3. **Program Utama (`src/main.cpp`)** Berkas ini bertindak sebagai titik masuk eksekusi program (*entry point*).
    - **Manajemen I/O**: Program diinstruksikan untuk menerima masukan langsung melalui antarmuka *Command Line Arguments* (`argc` dan `argv`), yang mana pengguna harus menspesifikasikan letak berkas sumber dan berkas tujuan keluaran.
    - **Alur Pemanggilan**: Program akan memanggil fungsi `tokenize()` dengan melempar parameter berkas *input*, lalu menampung hasilnya. Selanjutnya, program akan melakukan iterasi pada setiap token yang dihasilkan untuk menuliskannya ke dalam berkas *output* tujuan dengan memanggil metode `tokenTypeToString()`.

2.3 Alur Kerja Program

Alur kerja program pada lexer ini adalah dengan mengimplementasikan dalam bentuk DFA sebagai pembacaan dan tokenisasi pada setiap karakter yang dibaca oleh lexer. Pembuatan diagram keseluruhan DFA untuk lexer pada bahasa pemrograman Arion dapat dibagi menjadi bagian utama yang berisikan state-state untuk tokenisasi. Terdapat beberapa bagian yang dibuat, diantaranya:

- **Start state** merupakan kondisi awal pada DFA yang berfungsi sebagai titik mulai pembacaan karakter pertama dari aliran kode sumber (*source code*). *State* ini juga bertindak sebagai titik

kembali (*reset point*) sesaat setelah sebuah token berhasil dibentuk, untuk memulai pembacaan token berikutnya.

- **Indent state** merupakan *state* yang bertugas membaca dan membentuk token *identifier*, seperti nama variabel atau fungsi. *State* ini juga berfungsi sebagai jalur alternatif (*fallback*) apabila deretan karakter yang dibaca tidak sesuai dengan daftar kata kunci (*keyword*) baku pada bahasa pemrograman Arion. Indent state memiliki syarat bahwa karakter awal tidak boleh merupakan angka dan tidak ada pembacaan simbol di tengah string.
- **Keyword trie state** Merupakan sekumpulan transisi *state* terstruktur yang dikhususkan untuk mengidentifikasi dan membentuk token *keyword*. Proses pengenalan ini dirancang secara *case-insensitive*, sehingga dapat menerima masukan karakter huruf kapital ('A' hingga 'Z') maupun huruf kecil ('a' hingga 'z'). Dalam struktur ini, *final state* yang menandakan validitas sebuah *keyword* ditetapkan pada transisi karakter terakhir dari masing-masing kata kunci. Berikut adalah daftar *keyword* beserta tokennya yang terdapat pada bahasa pemrograman Arion:
    1. **and** : [andsy]
    2. **array** : [arraysy]
    3. **begin** : [beginsy]
    4. **case** : [casesy]
    5. **const** : [constsy]
    6. **div** : [idiv]
    7. **do** : [dosy]
    8. **downto** : [downtosy]
    9. **else** : [elsesy]
    10. **end** : [endsy]
    11. **for** : [forsy]
    12. **function** : [functionsy]
    13. **if** : [ifsy]
    14. **mod** : [imod]
    15. **not** : [notsy]
    16. **of** : [ofsy]
    17. **or** : [orsy]
    18. **procedure** : [proceduresy]
    19. **program** : [programsy]
    20. **record** : [recordsy]
    21. **repeat** : [repeatsy]
    22. **then** : [thensy]
    23. **to** : [tosy]
    24. **type** : [typesy]
    25. **until** : [untilsy]
    26. **var** : [varsy]
    27. **while** : [whilesy]

    Pada saat pembacaan karakter menelusuri cabang *Keyword Trie*, apabila karakter yang muncul tidak mengarah pada *keyword* yang valid, atau pembacaan karakter berakhir sebelum mencapai

final state dari suatu kata kunci, maka transisi *state* akan otomatis dialihkan ke dalam Ident State untuk diklasifikasikan sebagai *identifier*.

- **Numerical state** merupakan kumpulan transisi *state* yang bertugas membaca karakter digit dan membentuk token literal angka, yaitu bilangan bulat (intcon) dan bilangan riil (realcon). Saat *lexer* mendeteksi awalan digit, transisi akan diarahkan ke *state* bilangan bulat. Jika pada proses pembacaan tersebut ditemukan karakter titik desimal (.), DFA akan bertransisi menuju *state* bilangan riil. Namun, apabila setelah pembacaan angka selesai karakter berikutnya yang langsung mengikuti adalah sebuah huruf (tanpa spasi atau pemisah), maka DFA akan beralih ke *Unknown State* karena masukan tersebut tidak memenuhi aturan format angka yang valid.

- **Unknown state** merupakan *state* khusus yang bertindak sebagai penampung kesalahan (*error handler*) ketika *lexer* menemukan deretan karakter yang tidak valid atau tidak sesuai dengan aturan bahasa pemrograman Arion. *State* ini umumnya diaktifkan dalam dua kondisi:
  1. terjadinya kesalahan pembentukan token (*malformed token*), seperti ketika karakter awal adalah angka namun langsung diikuti oleh karakter huruf; dan
  2. ketika *lexer* membaca karakter atau simbol asing yang sama sekali tidak terdaftar dalam definisi token maupun operator Arion.
- **Symbols state** merupakan kumpulan transisi *state* yang bertugas membaca karakter-karakter simbol yang valid, meliputi berbagai operator (aritmatika, relasional, dan *assignment*) serta tanda baca (*delimiter*) pada bahasa pemrograman Arion. Dalam struktur ini, *final state* yang menandakan validitas sebuah simbol ditetapkan pada transisi karakter terakhir dari masing-masing simbol tersebut. Berikut adalah daftar simbol beserta tokennya yang terdapat pada bahasa pemrograman Arion:
  1. **+** : [plus]
  2. **-** : [minus]
  3. ***** : [times]
  4. **/** : [rdiv]
  5. **==** : [eql]
  6. **<>** : [neq]
  7. **>** : [gtr]
  8. **>=** : [geq]
  9. **<** : [lss]
  10. **<=** : [leq]
  11. **(** : [lparent]
  12. **)** : [rparent]
  13. **[** : [lbrack]
  14. **]** : [rbrack]
  15. **,** : [comma]
  16. **;** : [semicolon]
  17. **.** : [period]
  18. **:** : [colon]
  19. **:=** : [becomes]
- **Comment state** merupakan kumpulan transisi *state* khusus yang bertugas mendeteksi dan memproses blok komentar. *State* ini mulai aktif ketika *lexer* membaca karakter simbol pembuka

komentar, yaitu { atau (*. Selanjutnya, *lexer* akan membaca seluruh karakter di dalam blok tersebut hingga menemukan karakter simbol penutup yang bersesuaian, yaitu } atau *). *Final state* pada struktur ini tercapai tepat setelah karakter penutup berhasil dibaca, yang kemudian akan mengklasifikasikan keseluruhan blok tersebut menjadi satu token comment.

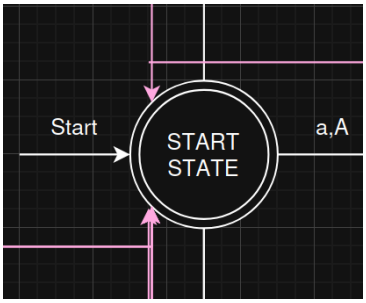
- **String and Character state** merupakan kumpulan transisi state khusus yang bertugas mendeteksi dan memproses blok string maupun karakter. State ini mulai aktif ketika *lexer* membaca karakter simbol pembuka berupa '. Selanjutnya, *lexer* akan membaca seluruh karakter di dalam blok tersebut hingga menemukan karakter simbol penutup yang bersesuaian, yaitu '. *Final state* pada struktur ini tercapai tepat setelah karakter penutup berhasil dibaca, yang kemudian akan mengklasifikasikan keseluruhan blok tersebut menjadi satu token string untuk isi dari karakter lebih dari 1 ataupun hanya karakter.

BAB 3
Implementasi

3.1 Implementasi Diagram Transisi DFA

Pada perancangan DFA, kami mengelompokkan masing masing jalur transisi DFA berdasarkan kategori token yang dikenali dalam bahasa pemrograman Arion.

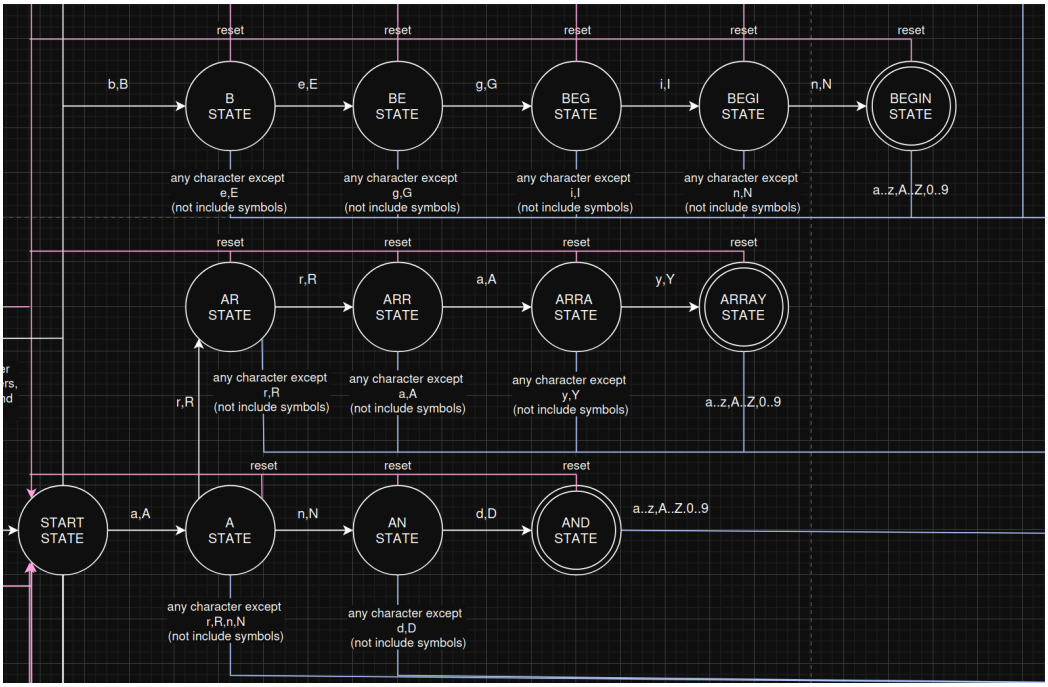
3.1.1 Diagram DFA Start State



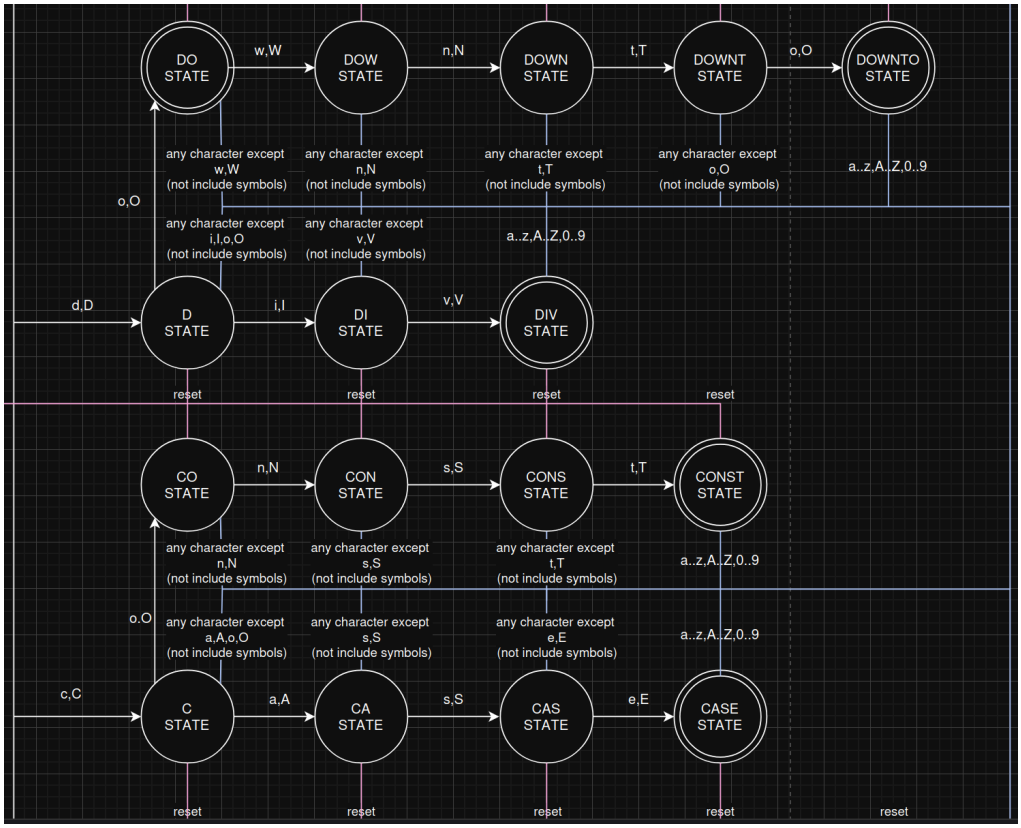
Gambar 3.1.1 Diagram DFA Start State

Titik awal seluruh proses adalah `START STATE`, yang menjadi titik masuk setiap kali mesin membaca karakter baru. Dari `START STATE`, mesin akan bertransisi ke jalur yang berbeda tergantung dari karakter pertama yang dibaca.

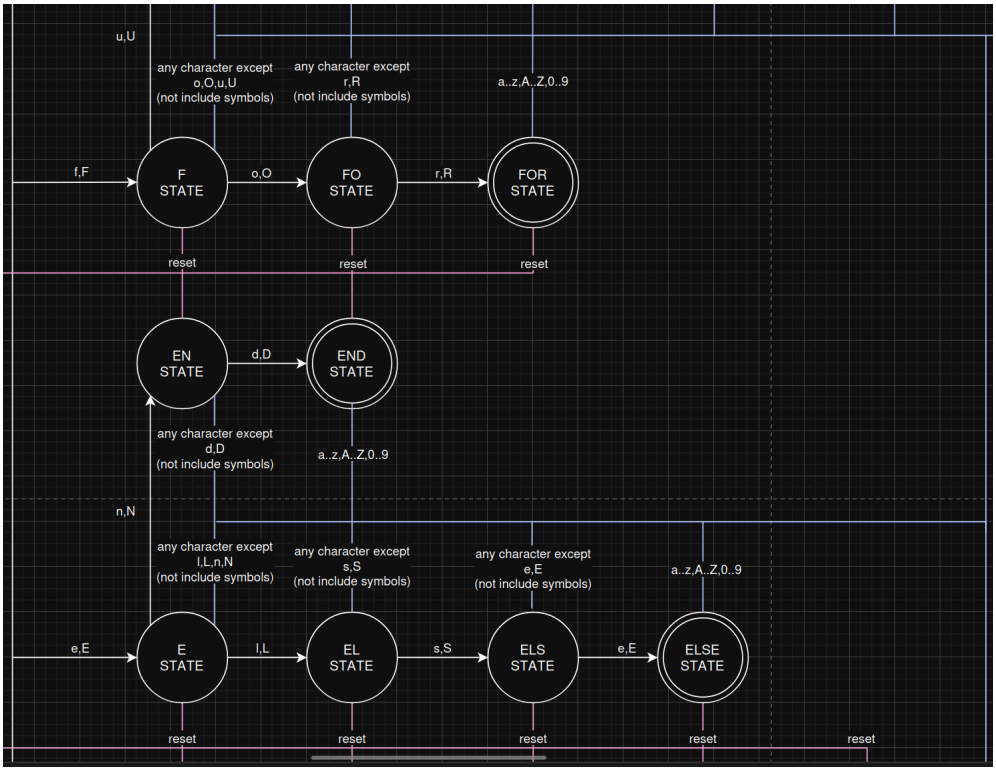
3.1.2 Diagram DFA Keyword Trie State



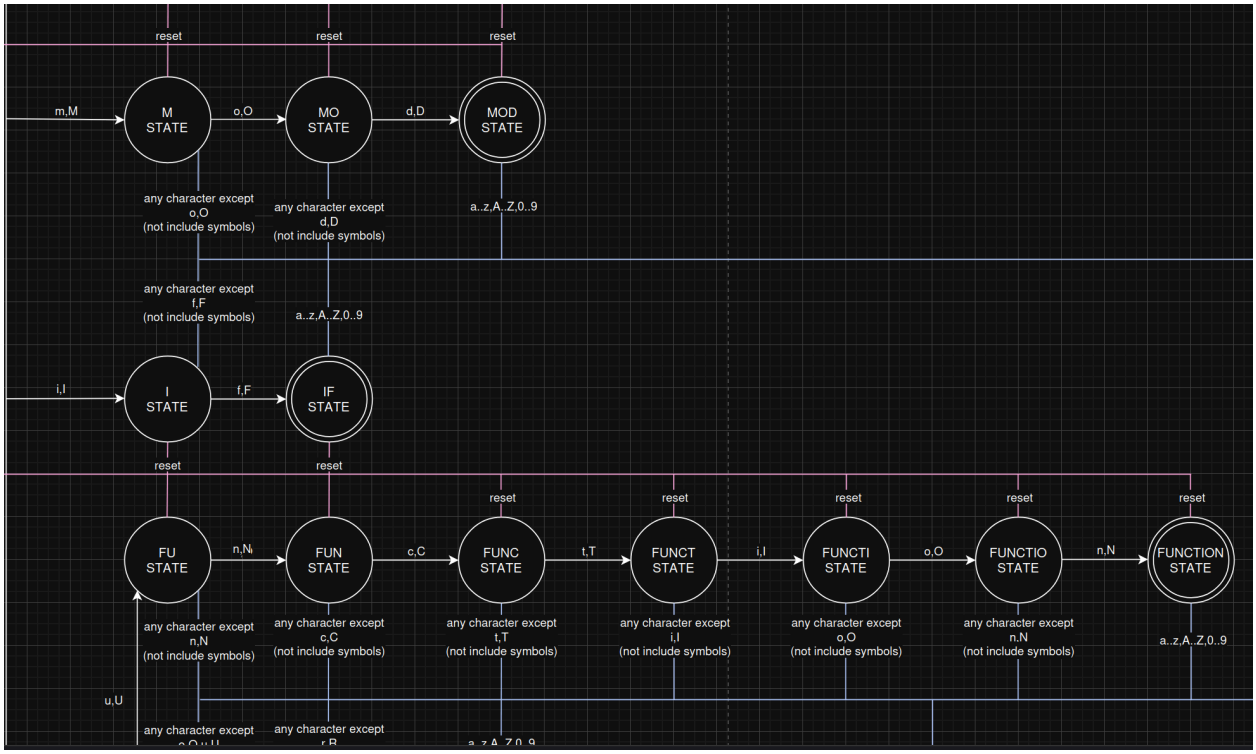
Gambar 3.1.2.1 Diagram DFA keyword trie state bagian 1



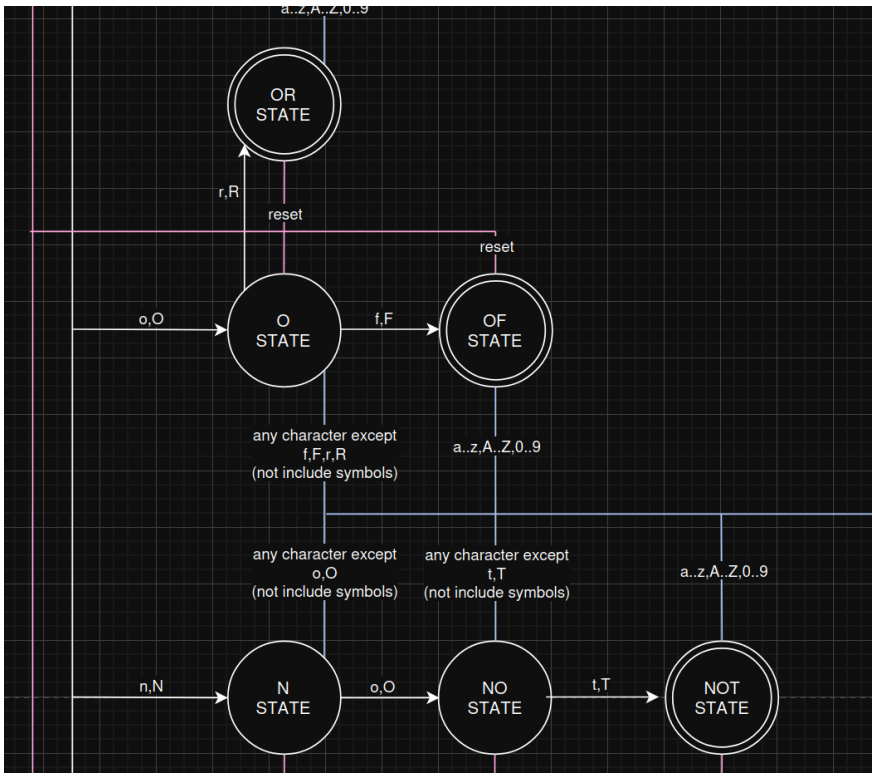
Gambar 3.1.2.2 Diagram DFA keyword trie state bagian 2



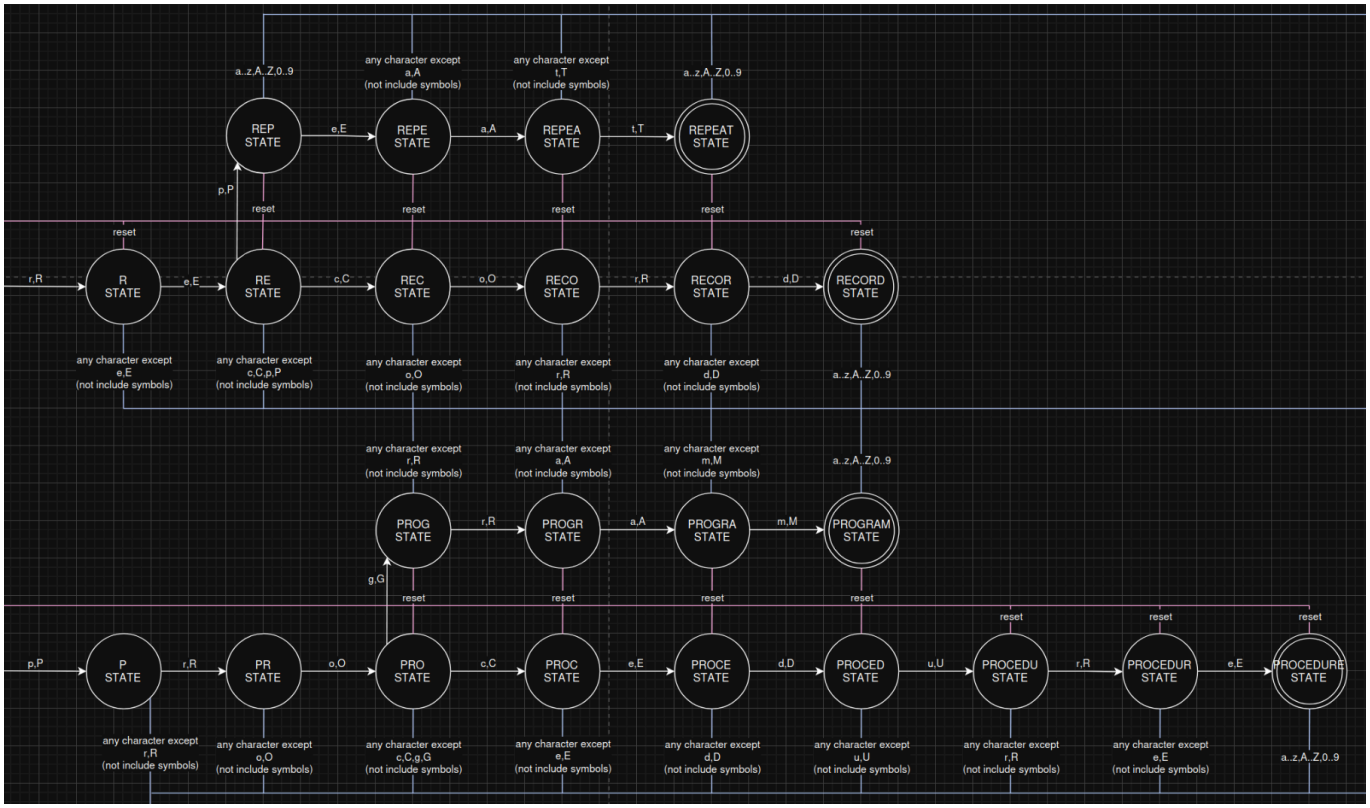
Gambar 3.1.2.3 Diagram DFA keyword trie state bagian 3



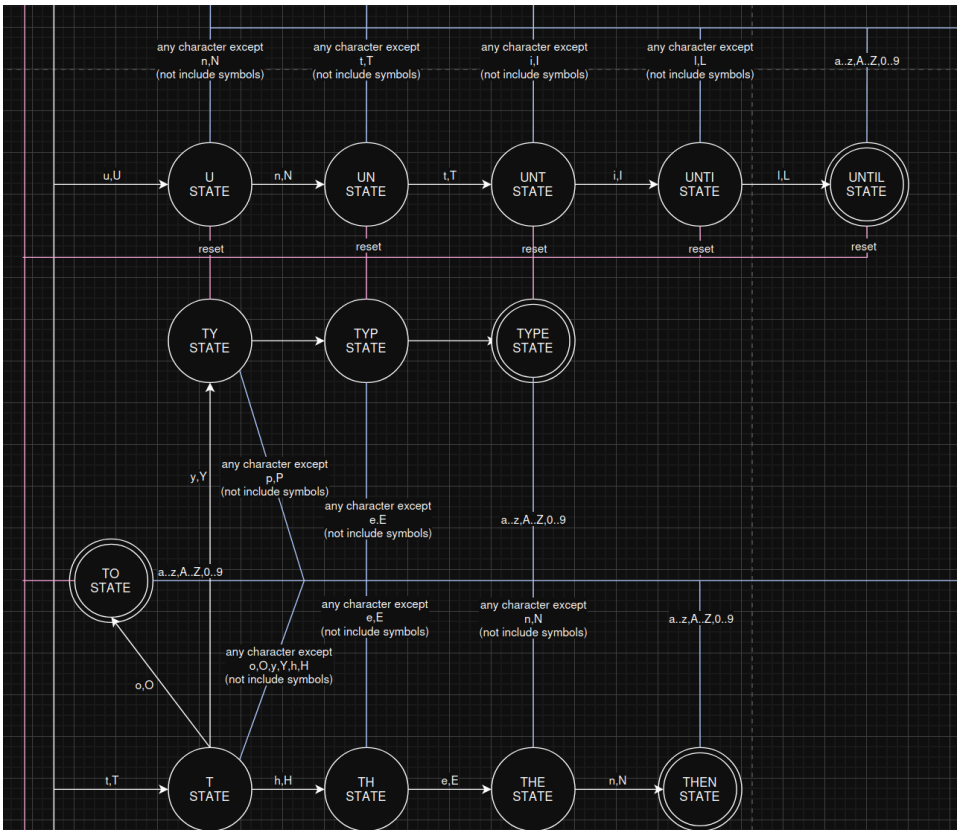
Gambar 3.1.2.4 Diagram DFA keyword trie state bagian 4



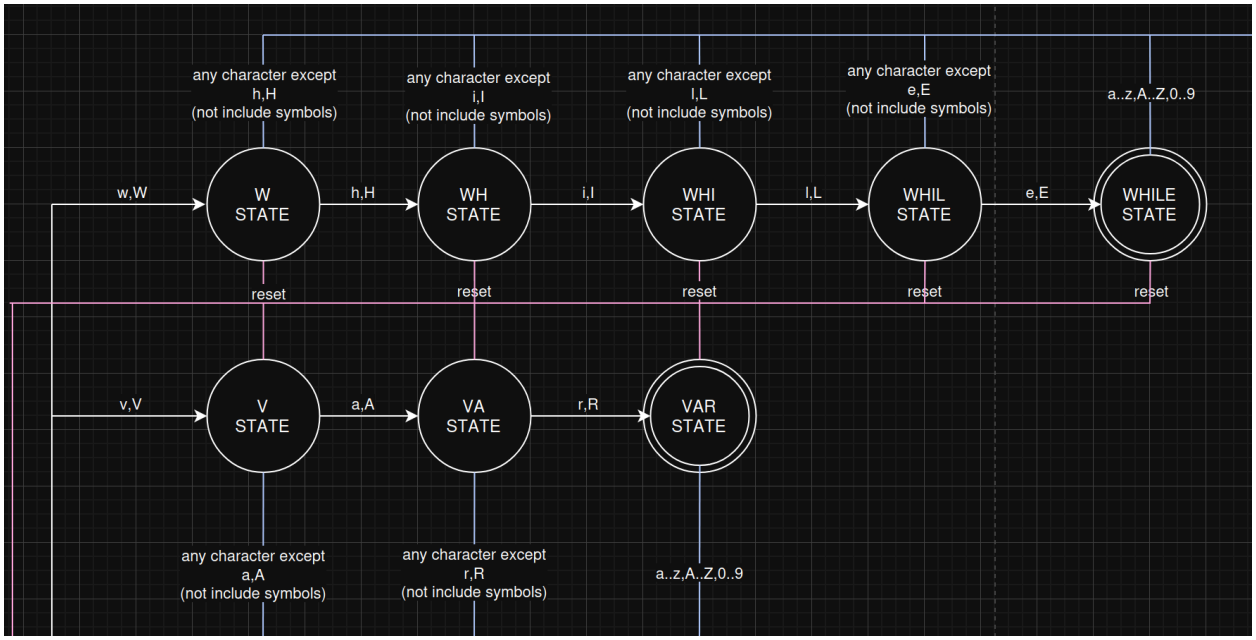
Gambar 3.1.2.5 Diagram DFA keyword trie state bagian 5



Gambar 3.1.2.6 Diagram DFA keyword trie state bagian 6



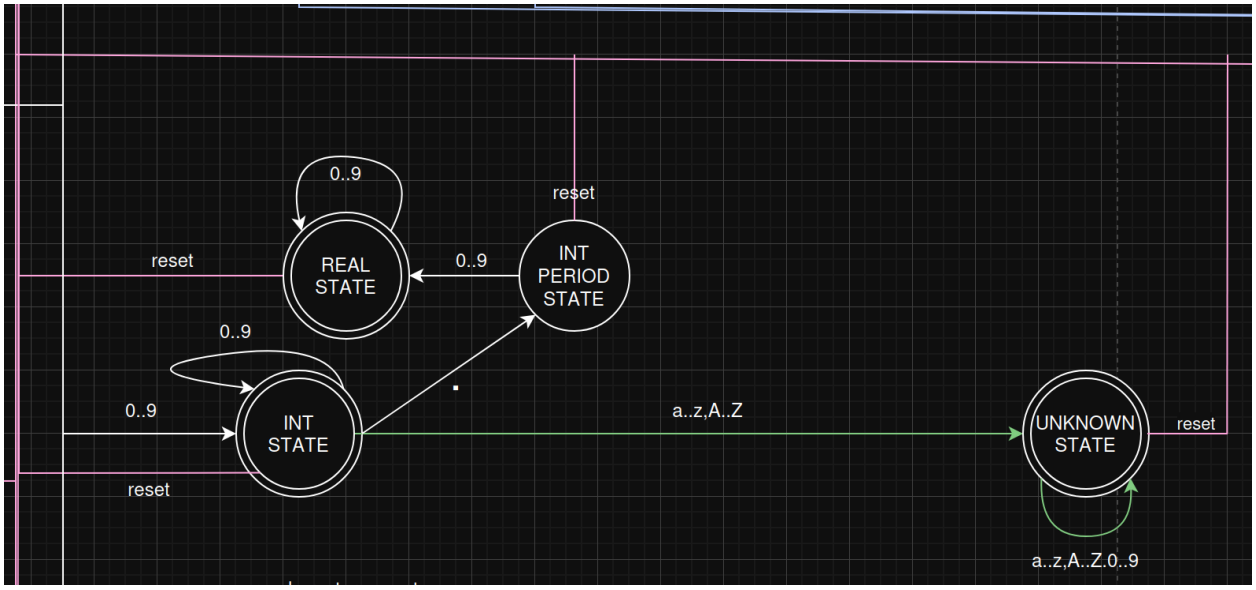
Gambar 3.1.2.7 Diagram DFA keyword trie state bagian 7



Gambar 3.1.2.8 Diagram DFA keyword trie state bagian 8

Untuk karakter huruf, mesin bertransisi ke state awal huruf yang bersesuaian dengan kategorinya, lalu mengikuti jalur trie huruf per huruf untuk mencocokkan keyword tersebut dengan token yang dikenal. Misalnya huruf pertama adalah a, maka mesin akan masuk ke `A STATE`, jika diteruskan dengan huruf n dan d, akan menjadi `AND STATE` (token).

3.1.3 Diagram DFA Numerical State

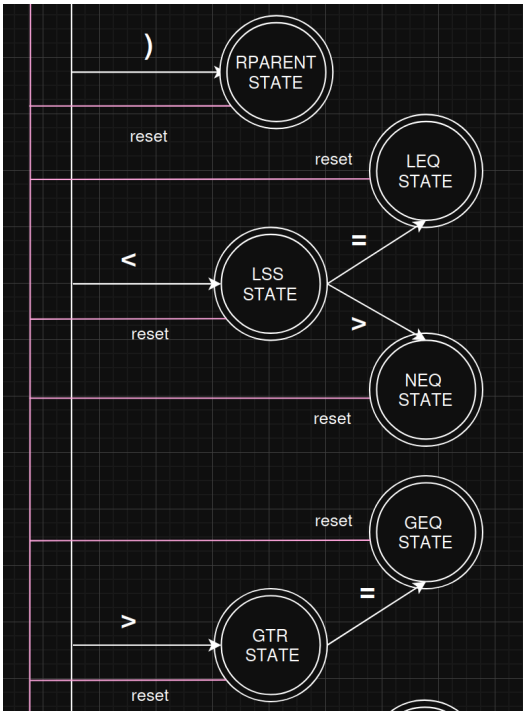


Gambar 3.1.3 Diagram DFA Numerical state

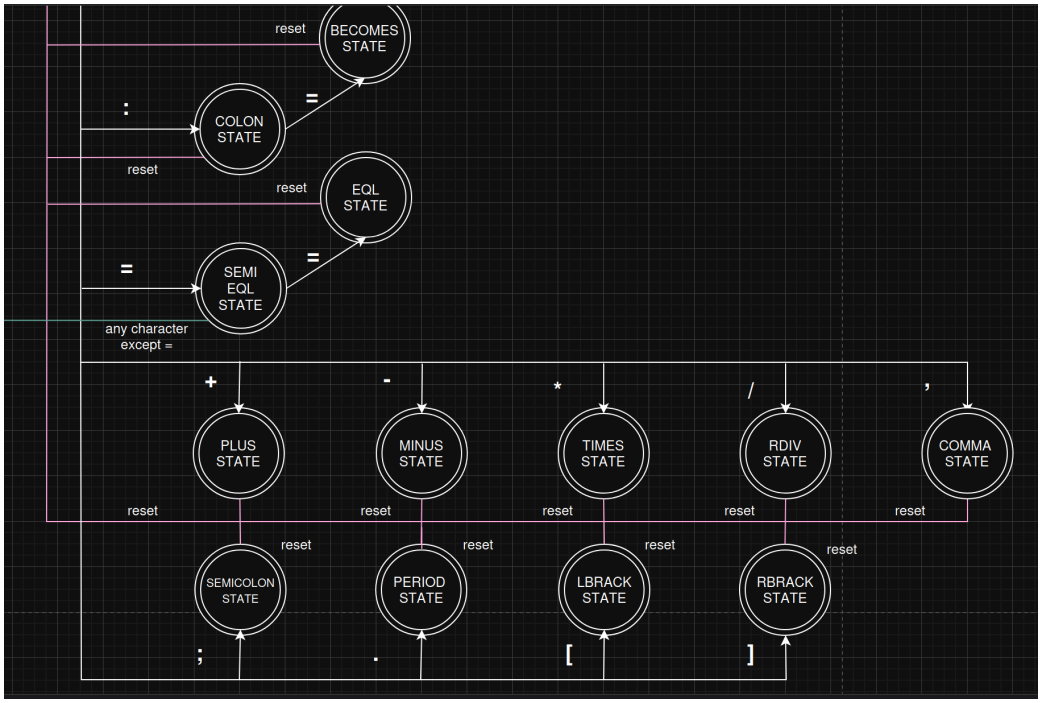
Untuk karakter digit 0 sampai 9, mesin bertransisi ke token int dan terus membaca digit selanjutnya. Jika ditemukan titik diikuti digit lain, mesin akan beralih ke token real, jika tidak, token yang

diemisikan adalah int dengan period (titik). Misal, 1.2 akan masuk ke `REAL STATE`, 1. Di `INT PERIOD STATE`, dan 1 di `INT STATE`.

3.1.4 Diagram DFA Symbols State



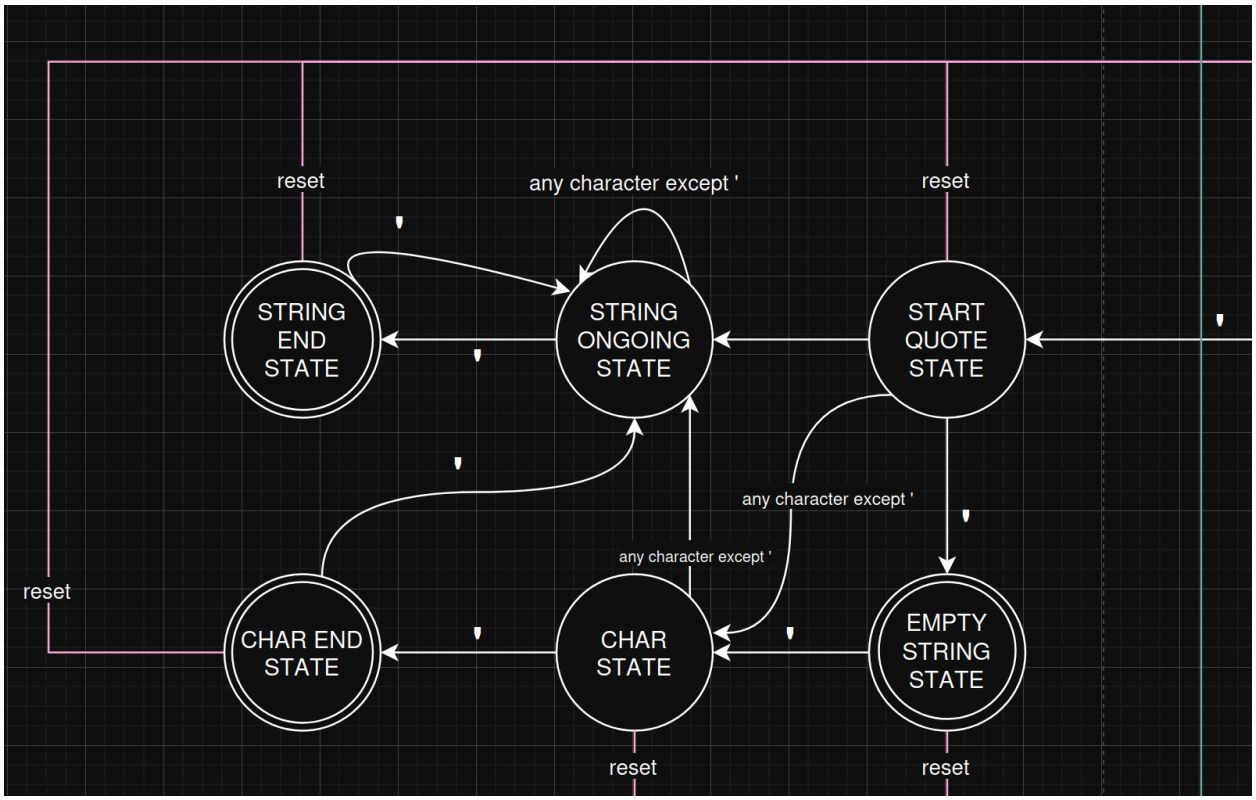
Gambar 3.1.4.1 Diagram DFA Symbols state bagian 1



Gambar 3.1.4.2 Diagram DFA Symbols state bagian 2

Untuk karakter simbol tunggal seperti +, -, *, dan serupa, mesin akan berpindah ke state simbol yang bersesuaian dan langsung menghasilkan token. Untuk karakter yang dapat membentuk token lebih dari satu karakter seperti <, >, dan :, mesin akan menunggu karakter berikutnya sebelum memutuskan token mana yang akan dihasilkan, misalnya simbol < diikuti = akan menghasilkan `LEQ STATE`, sedangkan jika diikuti > akan menghasilkan `NEQ STATE`.

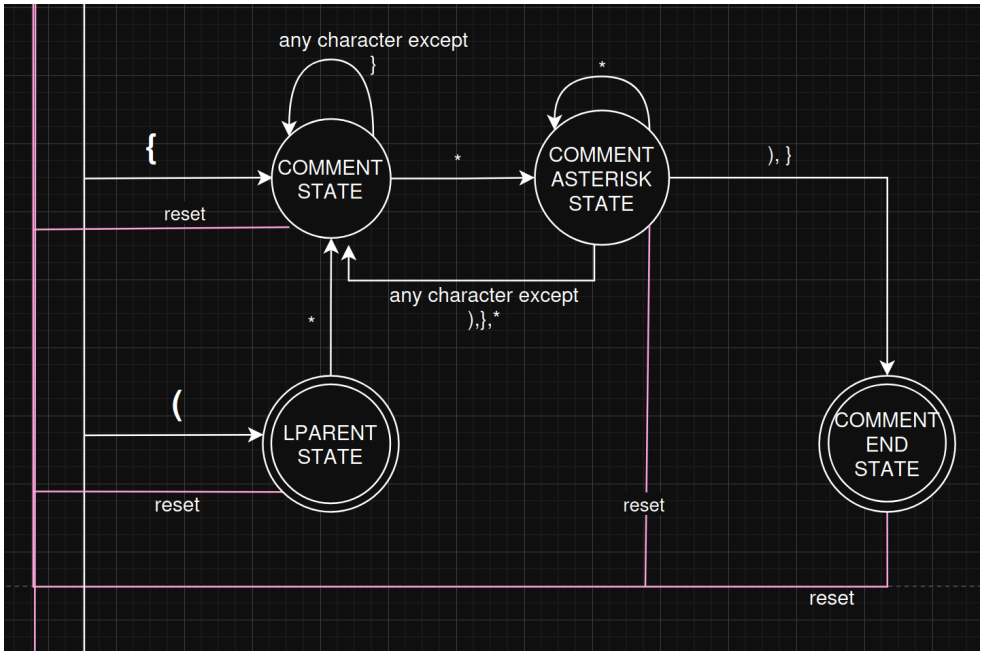
3.1.5 Diagram DFA String and Character State



Gambar 3.1.5 Diagram DFA String and Character state

Untuk karakter tanda kutip satu, mesin akan memasuki jalur pengenalan karakter string di `START QUOTE STATE`. Dari state ini, bercabang menjadi jalur char (satu karakter), string (lebih dari satu karakter), dan empty (nol karakter) tergantung dari berapa karakter yang diapit tanda kutip tunggal.

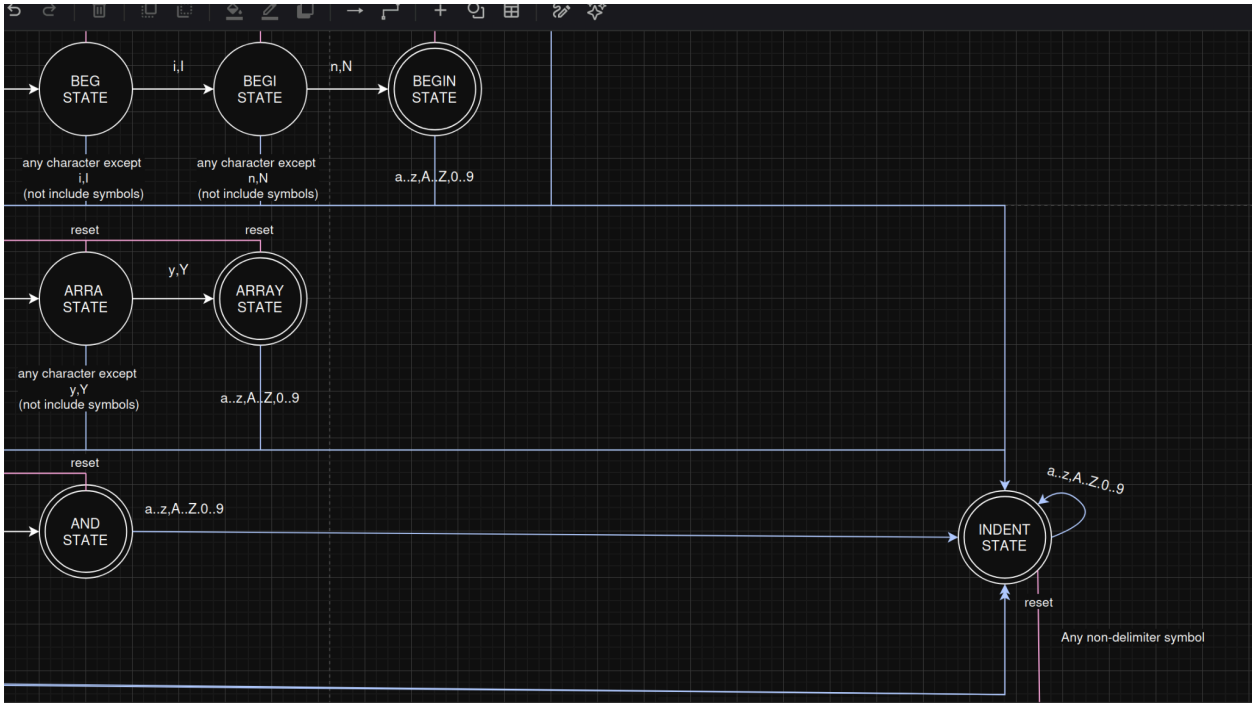
3.1.6 Diagram DFA Comment State



Gambar 3.1.6 Diagram DFA Comment state

Untuk karakter yang berhubungan dengan comment, mesin akan memasuki jalur pengenalan comment yang dapat berakhir di `COMMENT END STATE`. State mesin dimulai dari simbol { atau (yang diikuti dengan simbol bintang *, token yang dihasilkan adalah comment jika kurung tersebut ditutup dengan simbol } atau).

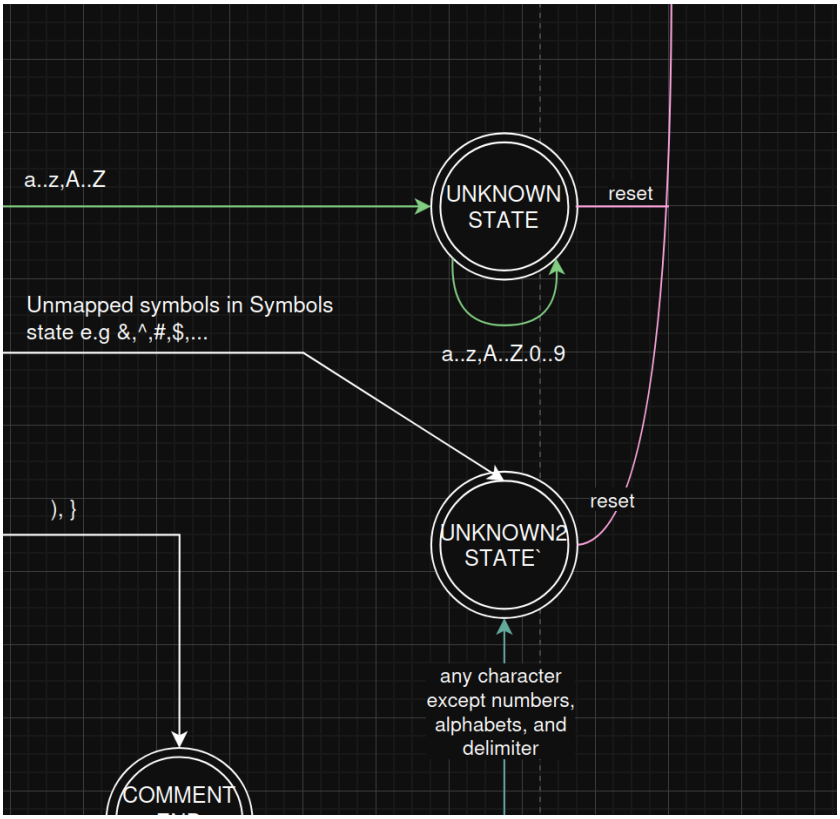
3.1.7 Diagram DFA Indent State



Gambar 3.1.7 Diagram DFA Indent state

Berhubungan dengan DFA Keyword Trie State, apabila di tengah pembacaan ditemukan karakter yang tidak sesuai dengan jalur token dikenal, maka susunan karakter tersebut akan dianggap sebagai token indent. Sebagai contoh, mesin sudah mencapai `AND STATE` (berarti token and), tetapi jika ditambah karakter alfanumerik lagi, susunan karakter bertransisi menjadi `INDENT STATE` karena susunan karakter tersebut sudah tidak dikenali lagi oleh Keyword Trie State.

3.1.8 Diagram DFA Unknown State



Gambar 3.1.8 Diagram DFA Unknown state

Untuk deretan karakter yang tidak valid dengan aturan bahasa pemrograman Arion, mesin akan berpindah ke `UNKNOWN STATE` atau `UNKNOWN2 STATE`. Unknown state dibagi menjadi dua agar dapat menampung 2 jenis deretan yang tidak valid:

1. `UNKNOWN STATE`: deretan angka yang diikuti dengan huruf
2. `UNKNOWN2 STATE`: karakter yang belum di mapping atau deretan simbol = diikuti oleh karakter selain angka, huruf, dan delimiter

3.2 Implementasi Modul Token dan State

Hasil akhir dari program ini merupakan kumpulan representasi token yang terbaca sebagai sebuah kelas `Token`. Kelas `Token` itu sendiri memiliki dua atribut yaitu, `type` yang berupa enumerasi `ListToken` dan merepresentasikan tipe dari token yang didapatkan, dan `value` yang berupa tipe `std::string` yang mengandung nilai sebuah token jika ia memilikinya (stringcon, charcon, intcon, realcon, comment, dan unknown). Semua enumerasi tipe token dapat dilihat di tabel token di bagian 2.1.

Method yang dimiliki oleh kelas Token ada dua yaitu constructor dan `tokenTypeToString` yang berfungsi mentranslasi token menjadi `std::string`. String yang dihasilkan akan berupa tipe token serta value jika tipe token berupa stringcon, charcon, intcon, realcon, comment, atau unknown.

Setiap state yang didalam diagram DFA jugalah direpresentasikan di program ini sebagai sebuah enumerasi tipe State. Penjelasan untuk setiap enumerasi state adalah sebagai berikut.

No.	**State**	**Penjelasan**	**Finish state?**	**Token yang dihasilkan apabila merupakan finish state.**
1	START_STATE	State dimana DFA mulai dan kembali saat terjadi tokenisasi	Ya	-
2	IDENT_STATE	State saat token yang sedang dibaca sudah terkonfirmasi sebagai ident	Ya	ident
3	COMMENT_STATE	State saat token yang sedang dibaca sudah terkonfirmasi sebagai isi komentar. DFA masuk ke state ini saat membaca karakter '{' saat di START_STATE atau karakter '*' saat ia di RPARENT_STATE	Bukan	-
4	COMMENT_ASTERISK_STATE	State saat DFA membaca karakter '*' saat sedang di COMMENT_STATE. State ini berfungsi agar DFA dapat membaca sekuens '*)'	Bukan	-
5	COMMENT_END_STATE	State saat DFA membaca karakter/sekuens penutup komentar yaitu '}' atau '*)'	Ya	comment
6	INT_STATE	State dimana DFA membaca karakter numerik setelah di START_STATE serta mempertahankan diri asalkan tetap menerima karakter numerik sebelum finish.	Ya	intcon
7	INT_PERIOD_STATE	State dimana DFA membaca karakter '.' saat ia sedang di INT_STATE. State ini berfungsi sebagai transisi dari INT_STATE ke REAL_STATE. Apabila ia merupakan finish state ia akan	Ya	intcon period

		mengemit dua token yaitu intcon dan period.		
8	REAL_STATE	State dimana DFA membaca karakter numerik saat ia sedang di INT_PERIOD_STATE, sehingga diinterpretasi menjadi sebuah sekuens bilangan real	Ya	realcon
9	START_QUOTE_STATE	State dimana DFA membaca karakter ‘’’ saat ia sedang di START_STATE	Bukan	-
10	STRING_ONGOING_STATE	State dimana DFA membaca sebuah karakter yang bukan ‘’’ saat ia sedang di CHAR_STATE atau saat DFA membaca sebuah karakter ‘’’ saat ia sedang di STRING_END_STATE atau CHAR_END_STATE. State ini menyatakan DFA sedang membaca sebuah nilai dari stringcon.	Bukan	-
11	STRING_END_STATE	State dimana DFA membaca sebuah karakter ‘’’ saat ia sedang di STRING_ONGOING_STATE.	Ya	stringcon
12	EMPTY_STRING_STATE	State dimana DFA membaca sebuah karakter ‘’’ saat ia sedang di START_QUOTE_STATE. State ini berfungsi untuk menghandle string yang masih kosong serta pembacaan karakter ‘’’	Ya	stringcon
13	CHAR_STATE	State dimana DFA membaca sebuah karakter yang bukan ‘’’ saat ia sedang di STRING_ONGOING_STATE. State ini menyatakan bahwa DFA sedang membaca sebuah nilai charcon.	Bukan	-
14	CHAR_END_STATE	State dimana DFA membaca sebuah karakter ‘’’ saat ia sedang di CHAR_STATE.	Ya	charcon
15	MINUS_STATE	State dimana DFA membaca karakter ‘-’ saat ia sedang di START_STATE	Ya	minus

16	PLUS_STATE	State dimana DFA membaca karakter ‘+’ saat ia sedang di START_STATE	Ya	plus
17	LPARENT_STATE	State dimana DFA membaca karakter ‘(’ saat ia sedang di START_STATE	Ya	lparent
18	RPARENT_STATE	State dimana DFA membaca karakter ‘)’ saat ia sedang di START_STATE	Ya	rparent
19	TIMES_STATE	State dimana DFA membaca karakter ‘*’ saat ia sedang di START_STATE	Ya	times
20	COMMA_STATE	State dimana DFA membaca karakter ‘,’ saat ia sedang di START_STATE	Ya	comma
21	PERIOD_STATE	State dimana DFA membaca karakter ‘.’ saat ia sedang di START_STATE	Ya	period
22	RDIV_STATE	State dimana DFA membaca karakter ‘/’ saat ia sedang di START_STATE	Ya	rdiv
23	COLON_STATE	State dimana DFA membaca karakter ‘:’ saat ia sedang di START_STATE	Ya	colon
24	BECOMES_STATE	State dimana DFA membaca karakter ‘=’ saat ia sedang di COLON_STATE	Ya	becomesy
25	LBRACK_STATE	State dimana DFA membaca karakter ‘[’ saat ia sedang di START_STATE	Ya	lbrack
26	RBRACK_STATE	State dimana DFA membaca karakter ‘]’ saat ia sedang di START_STATE	Ya	rbrack
27	SEMICOLON_STATE	State dimana DFA membaca karakter ‘;’ saat ia sedang di START_STATE	Ya	semicolon
28	LSS_STATE	State dimana DFA membaca karakter ‘<’ saat ia sedang di START_STATE	Ya	lss
29	LEQ_STATE	State dimana DFA membaca karakter ‘=’ saat ia sedang di LSS_STATE	Ya	leq
30	NEQ_STATE	State dimana DFA membaca karakter ‘>’ saat ia sedang di LSS_STATE	Ya	neq
31	SEMI_EQL_STATE	State dimana DFA membaca karakter ‘=’ saat ia sedang di START_STATE. State	Bukan	-

		ini berfungsi sebagai transisi dari START_STATE ke EQL_STATE		
32	EQL_STATE	State dimana DFA membaca karakter ‘=’ saat ia sedang di EQL_STATE	Ya	eql
33	GTR_STATE	State dimana DFA membaca karakter ‘>’ saat ia sedang di START_STATE	Ya	gtr
34	GEQ_STATE	State dimana DFA membaca karakter ‘=’ saat ia sedang di GTR_STATE	Ya	geq
35	A_STATE	State dimana DFA membaca karakter ‘A’ atau ‘a’ saat ia sedang di START_STATE	Bukan	-
36	AN_STATE	State dimana DFA membaca karakter ‘N’ atau ‘n’ saat ia sedang di A_STATE	Bukan	-
37	AND_STATE	State dimana DFA membaca karakter ‘D’ atau ‘d’ saat ia sedang di AN_STATE	Ya	andsy
38	AR_STATE	State dimana DFA membaca karakter ‘R’ atau ‘r’ saat ia sedang di A_STATE	Bukan	-
39	ARR_STATE	State dimana DFA membaca karakter ‘R’ atau ‘r’ saat ia sedang di AR_STATE	Bukan	-
40	ARRA_STATE	State dimana DFA membaca karakter ‘A’ atau ‘a’ saat ia sedang di ARR_STATE	Bukan	-
41	ARRAY_STATE	State dimana DFA membaca karakter ‘Y’ atau ‘y’ saat ia sedang di ARRAY_STATE	Ya	arraysy
42	B_STATE	State dimana DFA membaca karakter ‘B’ atau ‘b’ saat ia sedang di START_STATE	Bukan	-
43	BE_STATE	State dimana DFA membaca karakter ‘E’ atau ‘e’ saat ia sedang di B_STATE	Bukan	-
44	BEG_STATE	State dimana DFA membaca karakter ‘G’ atau ‘g’ saat ia sedang di BE_STATE	Bukan	-
45	BEGI_STATE	State dimana DFA membaca karakter ‘I’ atau ‘i’ saat ia sedang di BEG_STATE	Bukan	-
46	BEGIN_STATE	State dimana DFA membaca karakter ‘N’ atau ‘n’ saat ia sedang di BEGI_STATE	Ya	beginsy

47	C_STATE	State dimana DFA membaca karakter ‘C’ atau ‘c’ saat ia sedang di START_STATE	Bukan	-
48	CA_STATE	State dimana DFA membaca karakter ‘A’ atau ‘a’ saat ia sedang di C_STATE	Bukan	-
49	CAS_STATE	State dimana DFA membaca karakter ‘S’ atau ‘s’ saat ia sedang di CA_STATE	Bukan	-
50	CASE_STATE	State dimana DFA membaca karakter ‘E’ atau ‘e’ saat ia sedang di CAS_STATE	Ya	casesy
51	CO_STATE	State dimana DFA membaca karakter ‘O’ atau ‘o’ saat ia sedang di C_STATE	Bukan	-
52	CON_STATE	State dimana DFA membaca karakter ‘N’ atau ‘n’ saat ia sedang di CO_STATE	Bukan	-
53	CONS_STATE	State dimana DFA membaca karakter ‘S’ atau ‘s’ saat ia sedang di CON_STATE	Bukan	-
54	CONST_STATE	State dimana DFA membaca karakter ‘T’ atau ‘t’ saat ia sedang di CONS_STATE	Ya	constsy
55	D_STATE	State dimana DFA membaca karakter ‘D’ atau ‘d’ saat ia sedang di START_STATE	Bukan	-
56	DI_STATE	State dimana DFA membaca karakter ‘I’ atau ‘i’ saat ia sedang di D_STATE	Bukan	-
57	DIV_STATE	State dimana DFA membaca karakter ‘V’ atau ‘v’ saat ia sedang di DIV_STATE	Ya	idiv
58	DO_STATE	State dimana DFA membaca karakter ‘O’ atau ‘o’ saat ia sedang di D_STATE	Ya	dosy
59	DOW_STATE	State dimana DFA membaca karakter ‘W’ atau ‘w’ saat ia sedang di DO_STATE	Bukan	-
60	DOWN_STATE	State dimana DFA membaca karakter ‘N’ atau ‘n’ saat ia sedang di DOW_STATE	Bukan	-
61	DOWNT_STATE	State dimana DFA membaca karakter ‘T’ atau ‘t’ saat ia sedang di DOWN_STATE	Bukan	-
62	DOWNTO_STATE	State dimana DFA membaca karakter ‘O’	Ya	downtosy

		atau ‘o’ saat ia sedang di DOWNT_STATE		
63	E_STATE	State dimana DFA membaca karakter ‘E’ atau ‘e’ saat ia sedang di START_STATE	Bukan	-
64	EL_STATE	State dimana DFA membaca karakter ‘L’ atau ‘l’ saat ia sedang di E_STATE	Bukan	-
65	ELS_STATE	State dimana DFA membaca karakter ‘S’ atau ‘s’ saat ia sedang di EL_STATE	Bukan	-
66	ELSE_STATE	State dimana DFA membaca karakter ‘E’ atau ‘e’ saat ia sedang di ELS_STATE	Ya	elsesy
67	EN_STATE	State dimana DFA membaca karakter ‘N’ atau ‘n’ saat ia sedang di E_STATE	Bukan	-
68	END_STATE	State dimana DFA membaca karakter ‘D’ atau ‘d’ saat ia sedang di EN_STATE	Ya	endsy
69	F_STATE	State dimana DFA membaca karakter ‘F’ atau ‘f’ saat ia sedang di START_STATE	Bukan	-
70	FO_STATE	State dimana DFA membaca karakter ‘O’ atau ‘o’ saat ia sedang di F_STATE	Bukan	-
71	FOR_STATE	State dimana DFA membaca karakter ‘R’ atau ‘r’ saat ia sedang di FO_STATE	Ya	forsy
72	FU_STATE	State dimana DFA membaca karakter ‘U’ atau ‘u’ saat ia sedang di F_STATE	Bukan	-
73	FUN_STATE	State dimana DFA membaca karakter ‘N’ atau ‘n’ saat ia sedang di FU_STATE	Bukan	-
74	FUNC_STATE	State dimana DFA membaca karakter ‘C’ atau ‘c’ saat ia sedang di FUN_STATE	Bukan	-
75	FUNCT_STATE	State dimana DFA membaca karakter ‘T’ atau ‘t’ saat ia sedang di FUNC_STATE	Bukan	-
76	FUNCTI_STATE	State dimana DFA membaca karakter ‘I’ atau ‘i’ saat ia sedang di FUNCT_STATE	Bukan	-
77	FUNCTIO_STATE	State dimana DFA membaca karakter ‘O’ atau ‘o’ saat ia sedang di	Bukan	-

		FUNCTI_STATE		
78	FUNCTION_STATE	State dimana DFA membaca karakter ‘N’ atau ‘n’ saat ia sedang di FUNCTIO_STATE	Ya	functionsy
79	I_STATE	State dimana DFA membaca karakter ‘I’ atau ‘i’ saat ia sedang di START_STATE	Bukan	-
80	IF_STATE	State dimana DFA membaca karakter ‘F’ atau ‘f’ saat ia sedang di I_STATE	Ya	ifsy
81	M_STATE	State dimana DFA membaca karakter ‘M’ atau ‘m’ saat ia sedang di START_STATE	Bukan	-
82	MO_STATE	State dimana DFA membaca karakter ‘O’ atau ‘o’ saat ia sedang di M_STATE	Bukan	-
83	MOD_STATE	State dimana DFA membaca karakter ‘D’ atau ‘d’ saat ia sedang di MO_STATE	Ya	imod
84	N_STATE	State dimana DFA membaca karakter ‘N’ atau ‘n’ saat ia sedang di START_STATE	Bukan	-
85	NO_STATE	State dimana DFA membaca karakter ‘O’ atau ‘o’ saat ia sedang di N_STATE	Bukan	-
86	NOT_STATE	State dimana DFA membaca karakter ‘T’ atau ‘t’ saat ia sedang di NO_STATE	Ya	notsy
87	O_STATE	State dimana DFA membaca karakter ‘O’ atau ‘o’ saat ia sedang di START_STATE	Bukan	-
88	OF_STATE	State dimana DFA membaca karakter ‘F’ atau ‘f’ saat ia sedang di O_STATE	Ya	ofsy
89	OR_STATE	State dimana DFA membaca karakter ‘R’ atau ‘r’ saat ia sedang di O_STATE	Ya	orsy
90	P_STATE	State dimana DFA membaca karakter ‘P’ atau ‘p’ saat ia sedang di START_STATE	Bukan	-
91	PR_STATE	State dimana DFA membaca karakter ‘R’ atau ‘r’ saat ia sedang di P_STATE	Bukan	-
92	PRO_STATE	State dimana DFA membaca karakter ‘O’ atau ‘o’ saat ia sedang di PR_STATE	Bukan	-

93	PROC_STATE	State dimana DFA membaca karakter ‘C’ atau ‘c’ saat ia sedang di PRO_STATE	Bukan	-
94	PROCE_STATE	State dimana DFA membaca karakter ‘E’ atau ‘e’ saat ia sedang di PROC_STATE	Bukan	-
95	PROCED_STATE	State dimana DFA membaca karakter ‘D’ atau ‘d’ saat ia sedang di PROCE_STATE	Bukan	-
96	PROCEDU_STATE	State dimana DFA membaca karakter ‘U’ atau ‘u’ saat ia sedang di PROCED_STATE	Bukan	-
97	PROCEDUR_STATE	State dimana DFA membaca karakter ‘R’ atau ‘r’ saat ia sedang di PROCEDU_STATE	Bukan	-
98	PROCEDURE_STATE	State dimana DFA membaca karakter ‘E’ atau ‘E’ saat ia sedang di PROCEDUR_STATE	Ya	proceduresy
99	PROG_STATE	State dimana DFA membaca karakter ‘G’ atau ‘g’ saat ia sedang di PRO_STATE	Bukan	-
100	PROGR_STATE	State dimana DFA membaca karakter ‘R’ atau ‘r’ saat ia sedang di PROG_STATE	Bukan	-
101	PROGRA_STATE	State dimana DFA membaca karakter ‘A’ atau ‘a’ saat ia sedang di PROGR_STATE	Bukan	-
102	PROGRAM_STATE	State dimana DFA membaca karakter ‘M’ atau ‘m’ saat ia sedang di PROGRA_STATE	Ya	programsy
103	R_STATE	State dimana DFA membaca karakter ‘R’ atau ‘r’ saat ia sedang di START_STATE	Bukan	-
104	RE_STATE	State dimana DFA membaca karakter ‘E’ atau ‘e’ saat ia sedang di R_STATE	Bukan	-
105	REC_STATE	State dimana DFA membaca karakter ‘C’ atau ‘c’ saat ia sedang di RE_STATE	Bukan	-
106	RECO_STATE	State dimana DFA membaca karakter ‘O’ atau ‘o’ saat ia sedang di REC_STATE	Bukan	-

107	RECOR_STATE	State dimana DFA membaca karakter ‘R’ atau ‘r’ saat ia sedang di RECO_STATE	Bukan	-
108	RECORD_STATE	State dimana DFA membaca karakter ‘D’ atau ‘d’ saat ia sedang di RECOR_STATE	Ya	recordsy
109	REP_STATE	State dimana DFA membaca karakter ‘P’ atau ‘p’ saat ia sedang di RE_STATE	Bukan	-
110	REPE_STATE	State dimana DFA membaca karakter ‘E’ atau ‘e’ saat ia sedang di REP_STATE	Bukan	-
111	REPEA_STATE	State dimana DFA membaca karakter ‘A’ atau ‘a’ saat ia sedang di REPE_STATE	Bukan	-
112	REPEAT_STATE	State dimana DFA membaca karakter ‘T’ atau ‘t’ saat ia sedang di REPEA_STATE	Ya	repeatsy
113	T_STATE	State dimana DFA membaca karakter ‘T’ atau ‘t’ saat ia sedang di START_STATE	Bukan	-
114	TH_STATE	State dimana DFA membaca karakter ‘H’ atau ‘h’ saat ia sedang di T_STATE	Bukan	-
115	THE_STATE	State dimana DFA membaca karakter ‘E’ atau ‘e’ saat ia sedang di TH_STATE	Bukan	-
116	THEN_STATE	State dimana DFA membaca karakter ‘N’ atau ‘n’ saat ia sedang di THE_STATE	Ya	thensy
117	TY_STATE	State dimana DFA membaca karakter ‘Y’ atau ‘y’ saat ia sedang di T_STATE	Bukan	-
118	TYP_STATE	State dimana DFA membaca karakter ‘P’ atau ‘p’ saat ia sedang di TY_STATE	Bukan	-
119	TYPE_STATE	State dimana DFA membaca karakter ‘E’ atau ‘e’ saat ia sedang di TYPE_STATE	Ya	typesy
120	U_STATE	State dimana DFA membaca karakter ‘U’ atau ‘u’ saat ia sedang di START_STATE	Bukan	-
121	UN_STATE	State dimana DFA membaca karakter ‘N’ atau ‘n’ saat ia sedang di U_STATE	Bukan	-
122	UNT_STATE	State dimana DFA membaca karakter ‘T’ atau ‘t’ saat ia sedang di UN_STATE	Bukan	-

123	UNTI_STATE	State dimana DFA membaca karakter ‘I’ atau ‘i’ saat ia sedang di UNT_STATE	Bukan	-
124	UNTIL_STATE	State dimana DFA membaca karakter ‘L’ atau ‘l’ saat ia sedang di UNTIL_STATE	Ya	untilsy
125	V_STATE	State dimana DFA membaca karakter ‘V’ atau ‘v’ saat ia sedang di START_STATE	Bukan	-
126	VA_STATE	State dimana DFA membaca karakter ‘A’ atau ‘a’ saat ia sedang di V_STATE	Bukan	-
127	VAR_STATE	State dimana DFA membaca karakter ‘R’ atau ‘r’ saat ia sedang di VA_STATE	Ya	varsy
128	W_STATE	State dimana DFA membaca karakter ‘W’ atau ‘w’ saat ia sedang di START_STATE	Bukan	-
129	WH_STATE	State dimana DFA membaca karakter ‘H’ atau ‘h’ saat ia sedang di W_STATE	Bukan	-
130	WHI_STATE	State dimana DFA membaca karakter ‘I’ atau ‘i’ saat ia sedang di WH_STATE	Bukan	-
131	WHIL_STATE	State dimana DFA membaca karakter ‘L’ atau ‘l’ saat ia sedang di WHI_STATE	Bukan	-
132	WHILE_STATE	State dimana DFA membaca karakter ‘E’ atau ‘e’ saat ia sedang di WHIL_STATE	Ya	whilesy
133	UNKNOWN_STATE	State dimana DFA membaca karakter alfabet setelah saat ia sedang di INT_STATE atau REAL_STATE	Ya	unknown
134	UNKNOWN2_STATE	State dimana DFA membaca karakter/simbol yang tak dikenal pada saat START_STATE	Ya	unknown

3.3 Implementasi Logika DFA (Mesin Lexer)

Secara garis besar, proses Lexer di program ini dibangun menggunakan konsep Deterministic Finite Automata (DFA). Sistem kerjanya adalah dengan membaca source code karakter per karakter secara berurutan. Transisi perpindahan antar-state kita tangani menggunakan switch-case di dalam sebuah looping. Semuanya selalu berawal dari START_STATE setiap kali program mau membaca token baru.

Jika jalurnya udah sampai di state akhir yang menandakan tokennya valid, program akan memanggil fungsi lambda pushtoken buat menyimpan token tersebut, lalu statenya di-reset kembali ke START_STATE untuk lanjut mengevaluasi token selanjutnya.
Untuk mempermudah pemahaman, implementasi alur logika DFA ini dibagi ke dalam enam kategori utama berdasarkan jenis karakter awal yang dibaca:

1. Mekanisme Transisi dan Reset State

Salah satu aspek krusial dalam implementasi ini adalah peran karakter pembatas (simbol dan operator) dan spasi sebagai titik reset.

- Pada START_STATE: Karakter seperti spasi, tab, dan newline dianggap sebagai noise yang diabaikan (dikonsumsi tanpa diproses) untuk menjaga efisiensi memori.
- Pada State Lainnya: Begitu mesin berada dalam state selain START_STATE dan state untuk simbol ganda menemui karakter pembatas atau spasi, karakter tersebut bertindak sebagai titik reset. Mesin akan mengeksekusi fungsi pushtoken untuk membungkus akumulasi karakter sebelumnya, serta mengarahkan kembali mesin ke START_STATE agar karakter pembatas tersebut dapat diproses sebagai entitas token yang baru. Tetapi ada kasus khusus yang akan dibahas di kategori ke empat, yaitu jalur simbol.

2. Jalur Huruf: Membedakan Keyword dan Identifier

Jika karakter pertama (kondisi START_STATE) yang dibaca merupakan abjad (a-z atau A-Z), program tidak akan langsung menentukan itu sebagai identifier. Lexer akan mengecek dulu apakah susunan karakter ini merupakan bagian dari keyword bawaan (seperti begin, procedure, if, dsb). Implementasi teknis pada jalur ini mencakup tiga skenario utama dalam pembentukan token numerik:

- Transisi Karakter: Misalnya input pertama adalah 'p', state akan berpindah ke P_STATE. Kalau karakter selanjutnya 'r', lanjut ke PR_STATE, begitu terus mengikuti ejaan keyword yang ada.
- Case-Insensitive: Kita memanfaatkan fungsi tolower() supaya lexer tidak sensitif terhadap huruf besar-kecil. Jadi, penulisan PROCEDURE, procedure, atau ProCeDure akan tetap dibaca sebagai token yang sama.
- Dialihkan ke Identifier: Kalau di tengah proses pencocokan ternyata ejaannya tidak ada yang mengarah ke keyword yang ada (misalnya setelah PROC_ karakter selanjutnya malah huruf 'X', bukan 'E') atau karakter selanjutnya ternyata adalah angka, maka transisi keyword dibatalkan. Program akan langsung mengubah statenya menjadi IDENT_STATE dan akan menjadi identifier biasa. Disini karakter selanjutnya bisa berupa abjad maupun angka

3. Jalur Angka: Menangani Integer dan Real

Jalur ini otomatis terpanggil kalau karakter pertamanya (kondisi START_STATE) berupa angka (0-9). Di sini, lexer bertugas membedakan mana bilangan bulat (integer) dan mana yang bilangan desimal/riil (real). Implementasi teknis pada jalur ini mencakup tiga skenario utama dalam pembentukan token numerik:

- Integer: Awalnya, program masuk ke INT_STATE dan terus mengambil angka. Begitu bertemu delimiter seperti spasi atau simbol lain (kecuali titik), kumpulan angka tersebut langsung diresmikan menjadi token intcon.
- Mendeteksi Titik (Bilangan Real): Kalau saat berada di INT_STATE lexer membaca karakter titik (.), posisinya pindah ke INT_PERIOD_STATE. Di tahap ini, jika karakter selanjutnya adalah

angka lagi, berarti dipastikan ini bilangan real (pindah ke REAL_STATE). Tetapi, jika setelah titik ternyata bukan angka, nilai sebelumnya akan disimpan sebagai integer, dan titiknya dipisah menjadi token period.

- Setelah Angka Dilanjut Huruf: Untuk kasus ini, jika angka langsung diikuti dengan huruf (tanpa ada pembatas seperti spasi dll), maka terjadi perubahan state mennjadi UNKNOWN_STATE, yang dimana isi valuenya dimulai dari angka awal, sampai ditemukan batas kembali ke START_STATE.

4. Jalur Simbol: Operator dan Simbol

Melanjutkan kategori pertama, jika program bertemu dengan operator atau simbol tidak dikenal, program akan otomatis masuk ke jalur ini, dan token yang sebelumnya diakumulasi akan segera disimpan. Operator dan simbol yang teridentifikasi akan menjadi sebuah token dan menjadi pembatas, selain itu akan menjadi sebuah state UNKNOWN2_STATE dan menjadi pembatas. Implementasi teknis pada jalur ini memiliki 3 mekanisme pembacaan:

- Simbol Tunggal: Karakter seperti +, -, *, atau ; langsung dieksekusi menjadi token (misalnya dari PLUS_STATE langsung mengeluarkan plus).
- Simbol Ganda: Disinilah letak kasus khusus dari kategori pertama. Untuk simbol seperti =, :, <, atau >, program butuh mengecek satu karakter di depannya. Contohnya, kalau menemui :, program tidak langsung melakukan reset. Program akan masuk ke state khusus (dikasus ini COLON_STATE) untuk mengecek apakah karakter selanjutnya melengkapi simbol ganda tersebut atau tidak. Dari kategori pertama, diketahui bahwa setelah program membaca simbol, maka terjadi proses pushtoken dan reset state. Namun pada kasus ini, aturan tersebut sedikit dimodifikasi. Program akan menunda proses pushtoken sesaat untuk melakukan evaluasi karakter tambahan.
- Simbol Tidak Dikenal: Jika ditemukan simbol yang tidak terdaftar dalam spesifikasi bahasa (seperti @ atau #), karakter tersebut secara otomatis dilempar ke UNKNOWN2_STATE pada iterasi awal di START_STATE.

5. Jalur String dan Karakter

Saat membaca tanda petik tunggal ('), lexer akan mulai mengevaluasi bagian ini sebagai konstanta string atau karakter atau bahkan error. Implementasi teknis pada jalur ini memiliki 3 mekanisme pembacaan:

- Awalnya program masuk ke START_QUOTE_STATE. Jika langsung ditutup lagi ('), state-nya diarahkan ke EMPTY_STRING_STATE.
- Selama belum bertemu tanda petik penutup, karakter akan terus diakumulasi di dalam STRING_ONGOING_STATE.
- Menangani Error String: Misalnya kita membuat string tapi lupa memberikan petik penutup sampai ganti baris (bertemu newline \n). Supaya program tidak stuck mencari penutup, lexer akan langsung mengekstrak nilai yang ada dan memberikannya label token unknown.

6. Penanganan Komentar dan Mekanisme Keamanan (Unknown State)

Selain memproses token fungsional, lexer juga harus mampu menangani elemen non-instruksi dan input yang tidak valid. Terdapat 2 poin utama:

- Pengabaian Komentar: Teks yang berada di dalam kurung kurawal { } atau blok (* *) akan diserap sepenuhnya dan diabaikan dari proses pembentukan token semantik.
- Hierarki State Unknown: Terdapat dua tingkatan state untuk menangani karakter ilegal: UNKNOWN2_STATE untuk deteksi karakter asing, dan UNKNOWN_STATE untuk menangani karakter ilegal yang muncul di tengah pembentukan token yang berupa int. Mekanisme ini memastikan stabilitas mesin DFA sehingga program tidak berhenti secara mendadak (crash) saat menghadapi input yang tidak sesuai spesifikasi.
  Contoh:
    - UNKNOWN2_STATE: Jika input @ (langsung terbaca asing).
    - UNKNOWN_STATE: Jika input 123A (angka diikuti huruf secara ilegal).

Dengan logika transisi yang sudah dipetakan di atas, lexer ini diharapkan mampu menjalankan tugasnya untuk mengubah deretan karakter mentah menjadi kumpulan token yang bermakna. Implementasi ini menjadi pondasi penting bagi tahap analisis selanjutnya dalam proses kompilasi program yang sedang kita kembangkan.

3.4 Implementasi Program Utama (*Main*)

Implementasi program utama untuk milestone ini berada di `main.cpp`. Fungsi `main` di `main.cpp` menerima dua buah argumen saat melakukan run yaitu path dari file input dan path dari file output. Fungsi main akan meneruskan path file input yang akan dibaca oleh fungsi `tokenize` yang akan mengubahnya menjadi sebuah list objek `Token`. List `Token` tersebut lalu akan disalin di file output berdasarkan translasi dari setiap objek token ke `std::string` melalui method `tokenTypeToString`.

BAB 4
Pengujian

4.1 Pengujian Masukkan dan Keluaran Program

Masukkan	Keluaran

Test 1 - Nama File : input-1.txt
`{pengujian 1 - bagian tipe data primitif dan operator aritmatika}`
 	
`program Kalkulator;`
`var`
`  x, y  : integer;`
`  phi  : real;`
`  inisial : char;`
`  pesan  : string;`
`  hasil  : integer;`
`begin`
`  (* isi nilai awal untuk setiap variabel *)`
`  x  := 20;`
`  y  := 7;`
`  phi  := 3.14;`
`  inisial := 'A';`
`  pesan  := 'Halo Arion';`
 	
`  { hitung hasil penjumlahan, pengurangan, dan perkalian }`
`  hasil := x + y;`
`  hasil := x - y;`
`  hasil := x * y;`
 	
`  (* pembagian real: 22 dibagi 7 mendekati nilai phi *)`
`  phi := 22 / 7;`
 	
`  { tampilkan semua hasil ke layar }`
`  writeln(hasil);`
`  writeln(phi);`
`  writeln(inisial);`
`  writeln(pesan);`
`end.`
	Keluaran File :output-1.txt
	`comment (pengujian 1 - bagian tipe data primitif dan operator aritmatika)`
	`programsy`
	`ident (Kalkulator)`
	`semicolon`
	`varsy`
	`ident (x)`
	`comma`
	`ident (y)`
	`colon`
	`ident (integer)`
	`semicolon`
	`ident (phi)`
	`colon`
	`ident (real)`
	`semicolon`
	`ident (inisial)`
	`colon`
	`ident (char)`
	`semicolon`
	`ident (pesan)`
	`colon`
	`ident (string)`
	`semicolon`
	`ident (hasil)`
	`colon`
	`ident (integer)`
	`semicolon`
	`beginsy`
	`comment ( isi nilai awal untuk setiap variabel )`
	`ident (x)`
	`becomes`
	`intcon (20)`
	`semicolon`
	`ident (y)`
	`becomes`
	`intcon (7)`
	`semicolon`
	
	Seterusnya dapat dilihat di repository pada lampiran

Test 2 - Nama File :input-2.txt
`(* pengujian 2 - bagian percabangan dan operator logika *)`
 	
`program CekNilai;`
	Keluaran File :output-2.txt

```
var
    nilai : integer;
    lulus : integer;
begin
    nilai := 75;
    lulus := 0;

    { bandingkan nilai dengan beberapa ambang
batas menggunakan operator relasional }
    if nilai > 90 then
        writeln('Sangat Baik')
    else if nilai >= 75 then
        writeln('Baik')
    else if nilai < 50 then
        writeln('Kurang')
    else if nilai <= 59 then
        writeln('Cukup')
    else
        writeln('Sedang');

    (* pastikan nilai tidak sama dengan nol
sebelum diproses lebih lanjut *)
    if nilai <> 0 then
        writeln('Nilai tidak nol');

    { operator NOT membalik nilai logika dari
kondisi yang dibungkus kurung }
    if NOT (nilai < 0) then
        lulus := 1;

    (* operator AND mensyaratkan kedua sisi
bernilai benar secara bersamaan *)
    if (nilai >= 75) AND (nilai <= 100) then
        writeln('Lulus');

    { cek apakah nilai di luar rentang yang
diperbolehkan }
    if (nilai < 0) OR (nilai > 100) then
        writeln('Nilai tidak valid');

    (* operator == diuji di sini, perhatikan
jika ada bug pada EQL_STATE di lexer *)
    if nilai == 75 then
        writeln('Tepat 75');
end.
```

```
comment ( pengujian 2 - bagian percabangan dan operator logika )
programsy
ident (CekNilai)
semicolon
varsy
ident (nilai)
colon
ident (integer)
semicolon
ident (lulus)
colon
ident (integer)
semicolon
beginsy
ident (nilai)
becomes
intcon (75)
semicolon
ident (lulus)
becomes
intcon (0)
semicolon
comment ( bandingkan nilai dengan beberapa ambang batas menggunakan operator relasional
ifsy
ident (nilai)
gtr
intcon (90)
thensy
ident (writeln)
lparent
stringcon (Sangat Baik)
rparent
elsesy
ifsy
ident (nilai)
geq
intcon (75)
```

Seterusnya dapat dilihat di repository pada lampiran

Test 3 - Nama File :input-3.txt

```
{pengujian 3 - bagian perulangan dengan for,
while, dan repeat}

program Perulangan;
```

Keluaran File :output-3.txt

```
var
    i, n, total : integer;
begin
    n     := 5;
    total := 0;

    { jumlahkan angka 1 sampai n menggunakan
perulangan for naik }
    for i := 1 to n do
        total := total + i;

    writeln(total);

    (* cetak angka dari n turun ke 1
menggunakan for downto *)
    for i := n downto 1 do
        writeln(i);

    { perulangan while akan terus berjalan
selama kondisinya benar }
    i := 1;
    while i <= n do
    begin
        writeln(i);
        i := i + 1;
    end;

    (* repeat dijalankan minimal satu kali
sebelum kondisi until diperiksa }
    i := 1;
    repeat
        writeln(i);
        i := i + 1;
    until i > n;
end.
```

```
comment (pengujian 3 - bagian perulangan dengan for, while, dan repeat)
programsy
ident (Perulangan)
semicolon
varsy
ident (i)
comma
ident (n)
comma
ident (total)
colon
ident (integer)
semicolon
beginsy
ident (n)
becomes
intcon (5)
semicolon
ident (total)
becomes
intcon (0)
semicolon
comment ( jumlahkan angka 1 sampai n menggunakan perulangan for naik )
forsy
ident (i)
becomes
intcon (1)
tosy
ident (n)
dosy
ident (total)
becomes
ident (total)
plus
ident (i)
semicolon
ident (writeln)
```

Seterusnya dapat dilihat di repository pada lampiran

Test 4 - Nama File :input-4.txt

```
(* pengujian 4 - bagian deklarasi tipe
kompleks, fungsi, dan prosedur *)

program TipeKompleks;
const
    MAX := 10;
    PI  := 3.14159;
type
    Titik : record
        x : real;
        y : real;
    end;
    Daftar : array [1..10] of integer;
var
```

Keluaran File :output-4.txt

```
    p    : Titik;
    data : Daftar;
    i    : integer;

{ fungsi ini menerima satu bilangan bulat dan
mengembalikan kuadratnya }
function Kuadrat(n : integer) : integer;
begin
    Kuadrat := n * n;
end;

(* prosedur ini mengisi array dengan nilai
kuadrat dari indeksnya *)
procedure IsiArray(var arr : Daftar; n :
integer);
var
    j : integer;
begin
    for j := 1 to n do
        arr[j] := j * j;
end;

begin
    { isi koordinat titik dengan nilai real }
    p.x := 3.0;
    p.y := 4.0;

    (* panggil prosedur untuk mengisi data,
lalu tampilkan hasilnya *)
    IsiArray(data, MAX);

    for i := 1 to MAX do
        writeln(data[i]);

    writeln(Kuadrat(5));
end.
```

```
comment ( pengujian 4 - bagian deklarasi tipe kompleks, fungsi, dan prosedur )
programsy
ident (TipeKompleks)
semicolon
constsy
ident (MAX)
becomes
intcon (10)
semicolon
ident (PI)
becomes
realcon (3.14159)
semicolon
typesy
ident (Titik)
colon
recordsy
ident (x)
colon
ident (real)
semicolon
ident (y)
colon
ident (real)
semicolon
endsy
semicolon
ident (Daftar)
colon
arraysy
lbrack
intcon (1)
period
period
intcon (10)
rbrack
ofsy
```

Seterusnya dapat dilihat di repository pada lampiran

Test 5 - Nama File :input-5.txt

```
{pengujian 5 - bagian case, operator div dan
mod, serta variasi komentar}

program KasusLengkap;
var
    angka   : integer;
    bagi    : integer;
    sisa    : integer;
    harakat : char;
begin
    angka := 17;

    { bagi integer: 17 dibagi 5 hasilnya 3,
```

Keluaran File :output-5.txt

```
bukan 3.4 }
    bagi := angka div 5;

    (* sisa bagi: 17 mod 5 hasilnya 2 karena 5
masuk 3 kali dengan sisa 2 *)
    sisa := angka MOD 5;

    writeln(bagi);
    writeln(sisa);

    { gunakan case of untuk memeriksa nilai
sisa pembagian satu per satu }
    case sisa of
        0 : writeln('Habis dibagi 5');
        1 : writeln('Sisa 1');
        2 : writeln('Sisa 2');
        3 : writeln('Sisa 3');
        4 : writeln('Sisa 4');
    end;

    (* simpan satu karakter huruf ke dalam
variabel bertipe char *)
    harakat := 'Z';

    writeln(harakat);
end.
```

```
comment (pengujian 5 - bagian case, operator div dan mod, serta variasi komentar)
programsy
ident (KasusLengkap)
semicolon
varsy
ident (angka)
colon
ident (integer)
semicolon
ident (bagi)
colon
ident (integer)
semicolon
ident (sisa)
colon
ident (integer)
semicolon
ident (harakat)
colon
ident (char)
semicolon
beginsy
ident (angka)
becomes
intcon (17)
semicolon
comment ( bagi integer: 17 dibagi 5 hasilnya 3, bukan 3.4 )
ident (bagi)
becomes
ident (angka)
div
intcon (5)
semicolon
comment ( sisa bagi: 17 mod 5 hasilnya 2 karena 5 masuk 3 kali dengan sisa 2 )
ident (sisa)
becomes
ident (angka)
```

Seterusnya dapat dilihat di repository pada lampiran

BAB 5
Kesimpulan dan Saran

5.1 Kesimpulan

Tahap *lexical analysis* merupakan langkah awal yang sangat penting dalam pembuatan compiler pada pembuatan suatu bahasa pemrograman yang ada. Proses ini bertugas untuk memindai urutan karakter yang ada pada file untuk dikelompokkan menjadi rangkaian token yang memiliki makna semantik. Alat *lexical analysis* atau *lexer* dibuat dengan menerapkan konsep Deterministic Finite Automaton (DFA), yaitu sebuah model komputasi automata yang membaca setiap karakter masukan dan memicu transisi yang pasti dari satu state ke state lainnya hingga mencapai sebuah *finish state* atau *accepting state*.

Keberhasilan memodelkan *lexer* dengan DFA memastikan proses pemindahan berjalan sangat efisien dan tanpa ambiguitas. Hasil analisis leksikal ini memegang peranan krusial untuk tahapan kompilasi berikutnya. Dengan menyaring dan mengelompokkan bagian dari elemen-elemen untuk akan diproses kedepannya dalam prosesi tahap *syntax analysis* atau parsing yang berfokus pada pengecekan struktur tata bahasa pemrograman yang memudahkan dalam *semantic analysis* dan *code generation*.

5.2 Saran

Dalam pengembangan dan iterasi milestone berikutnya, perancangan arsitektur sebaiknya memprioritaskan pendefinisian dan analisis keyword serta token secara komprehensif sejak awal pembentukan konsep, sebelum melangkah jauh ke tahap implementasi kode. Merencanakan spesifikasi masalah dengan baik dari awal akan mempermudah proses konstruksi diagram untuk lexer untuk bisa membagi dan memisahkan setiap deretan karakter ke token-token masing-masing dan memudahkan proses tahapan kedepannya. Pemecahan dan membagi token-token ke dalam kelompok kategori yang spesifik, seperti kelompok simbol, keyword, dan lainnya, akan menjadi membuat lebih jelas, mudah divisualisasikan dan meminimalisir potensi terjadinya kesalahan kedepannya.

Lampiran

Link Github : https://github.com/Alpaomega1136/JML-Tubes-IF2224-2026.git
Link Diagram DFA lexer :
https://app.diagrams.net/#G1cUoIh2k24-wSg9v2nve_WYQwX2OiRVSJ#%7B%22pageId%22%3A%22qTB-OnfOO5RK6jn4iWMg%22%7D
Pembagian Tugas :

NIM	Nama	Pengerjaan pada milestone ini
13524015	Mahatma Brahmana	Membuat diagram DFA & membuat kode lexer
13524057	Benedict Darrel Setiawan	Membuat diagram DFA & membuat kode lexer
13524059	Raymond Jonathan Dwi Putra J	Membuat diagram DFA & membuat laporan
13524111	Reynard Anderson Wijaya	Membuat diagram DFA & membuat laporan

Daftar Pustaka

Tutorialspoint. (2014). *Compiler design – lexical analysis*. Diakses dari: https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm (PDF4PRO)

Introduction to Automata Theory, Languages, and Computation. (2014). *Introduction to automata theory, languages, and computation* (3rd ed.). Pearson. (perpustakaan.itera.ac.id)

Compilers: Principles, Techniques, and Tools. (2006). *Compilers: Principles, techniques, and tools*. Pearson.

GNU Project. (n.d.). *An introduction to makefiles*. Diakses dari: https://www.gnu.org/software/make/manual/html_node/Introduction.html

Hackthology. (n.d.). *Faster tokenization with a DFA backend for lexmachine*. Diakses dari: https://hackthology.com/faster-tokenization-with-a-dfa-backend-for-lexmachine.html